

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

September 8, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	September 8, 2026	55 pages, 12 appendix pages

Tiivistelmä — Referat — Abstract

Context. Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

Objective. The thesis compares modern C++ and Rust on a user equipment (UE) Context Management service modelled on a real Nokia 5G base-station control-plane component (gNB-CU-CP). Average-throughput comparisons leave four dimensions open: tail latency, the C++ defect classes Rust prevents at compile time, the overhead of its safety model, and the cost of incremental adoption.

Methods. One language-agnostic specification is frozen first, then implemented twice with identical algorithms, concurrency model, allocator, and build settings. A synthetic workload is swept to overload, both versions pass the same parity tests, and the evaluation adds fault injection and a foreign function interface (FFI) prototype.

Results. Safe Rust rejects all seven injected memory-safety and data-race defect classes at compile time. In C++ the same seven build cleanly and show up only at runtime, if a sanitiser catches them. Seven further classes fail in both languages. On two of them C++ tooling reports what Rust does not. The seam to C++ leaves the core `unsafe`-free and costs at most 10.91% of one operation. Performance changes at saturation. Below it the test resolves a few microseconds in Rust's favour, yet an end-to-end call spends only 0.250% on language work. Cross-thread handoff takes the rest, so the sweep reflects the design more than the language. Timed alone, Rust's operation is 1.6× faster. The Rust arms still meet their offered rate to 448 kops/s, the C++ arms only to 392 kops/s. After both have stopped tracking, C++ holds 1.7× the throughput of Rust in 20/20 interleaved repeats with both allocators.

Conclusions. The main cost of adopting Rust is engineering effort and the learning curve, not run-time performance at nominal load. The gain is freedom from data races, not from concurrency defects in general. A nominal-load test would not expose the gap.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures **iii**

List of Tables **iv**

List of acronyms **v**

1 Introduction **1**

1.1 Motivation and industrial context 1

1.2 Problem statement 4

1.3 Research questions and hypotheses 5

2 Background: Systems Languages for Latency-Critical Software **6**

2.1 RAN control-plane systems and the gNB-CU-CP 6

2.2 Predictability and the two languages 7

2.3 Research gap 7

3 Research Methodology **10**

3.1 The UE Context Management subsystem 10

3.2 Research approach 11

3.3 Methods per research question, in outline 13

3.4 Fairness as a controlled variable 14

3.5 Statistical treatment 14

3.6 Reproducibility and research artifacts 14

3.7 The two implementations in brief 15

4 Experimental Setup and Evaluation **16**

4.1 Testbed configuration 16

4.2 Workload and evaluation measurements 17

4.3 Evaluation data 18

4.4 Evaluation results 18

4.4.1 Below saturation (RQ1, RQ3) 19

4.4.2 Past saturation (RQ1) . . . 26
4.4.3 Resource use (RQ3) . . . 29
4.4.4 Safety via fault injection (RQ2) . . . 31
4.4.5 Migration (RQ4) . . . 34

5 Discussion **38**
5.1 Summary of findings . . . 38
5.2 Discussion on the research questions . . . 40
5.3 Revisiting the hypotheses . . . 44
5.4 Engineering guidelines for incremental adoption . . . 45
5.5 Where this study sits, and what it cannot support . . . 45
5.6 Ethical and professional reflection . . . 46

6 Conclusion and Future Work **47**

Statement on the Use of AI Tools **49**

Bibliography **51**

A Key terms

B Literature search protocol

C Fault-injection example

D Statistical design: family membership and resolving power

E Host and campaign configuration

F Implementation of the two cores

G Engineering guidelines for incremental adoption

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 6
2.2 What the neighbouring studies control, and what they leave varying 8

3.1 UE-context lifecycle state machine 10

4.1 The two workloads, and the lifecycle the mixed one walks 17
4.2 The tested family drawn as paired effects 22
4.3 The end-to-end operation decomposed, against what the sweep can resolve 24
4.4 The load sweep: tail latency and achieved throughput 25
4.5 Latency across the percentile range, and how far it resolves 26
4.6 Divergence past saturation 27
4.7 Resource use, and the two effects inside peak RSS 30
4.8 Where the guarantee stops: the outcome matrix 35
4.9 The C++-to-Rust FFI migration seam 36
4.10 The seam's cost on one operation, against its own uncertainty 37

5.1 Where Rust's compile-time guarantee stops 42

List of Tables

3.1 Research design: how each research question is answered 13

4.1 Per-operation latency at the representative load 20
4.2 Exact paired permutation test of C++ against Rust 21
4.3 How large each effect could be 22
4.4 Where an end-to-end operation's cost actually goes 23
4.5 What each variant sustains past saturation 27
4.7 Throughput through the contended synchronisation primitives alone 28
4.6 Exact paired test past saturation 28
4.8 Resource use at the representative load 30
4.9 Defect-class taxonomy: measured outcomes 34
4.10 FFI migration prototype: seam size, safety, and cost 36

5.1 Summary of findings per research question 39

A.1 Key terms i

E.1 The three measurement hosts ii
E.2 Configuration of the service, the workload, and the campaign iii
E.3 Inter-operation jitter at the representative load iv
E.4 The resident set decomposed by the allocator iv

List of acronyms

Each acronym below is also written out where the text first uses it. Terms that are not acronyms, such as *tail latency* and *shard*, are collected in Appendix A.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
ACM	Association for Computing Machinery
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
ASan	AddressSanitizer
ASN.1	Abstract Syntax Notation One
AWS	Amazon Web Services
CISA	Cybersecurity and Infrastructure Security Agency
CPU	Central processing unit
CVE	Common Vulnerabilities and Exposures
DBLP	The computer science bibliography of that name
DiVA	Digitala Vetenskapliga Arkivet, the Nordic research repository
DPDK	Data Plane Development Kit
EU	European Union
FBI	Federal Bureau of Investigation
FFI	Foreign function interface
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane: the system class studied here
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IEEE	Institute of Electrical and Electronics Engineers
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
JSON	JavaScript Object Notation
KPI	Key performance indicator
NG-RAN	Next-generation radio access network
NUMA	Non-uniform memory access
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RFC	Request for Comments
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size

Acronym	Expansion
SMT	Simultaneous multithreading
TEID	Tunnel endpoint identifier
TSan	ThreadSanitizer
UBSan	UndefinedBehaviorSanitizer
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here
USENIX	The USENIX Association

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers, and the radio access network (RAN), long the most hardware-bound part of the stack, is part of that shift. In 5G the base station, formally the next-generation NodeB (gNB), splits into a central unit and one or more distributed units, and the central unit splits again into control-plane and user-plane functions [1, 2]. Open-RAN pushes the idea further, into virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), is a large, stateful, highly concurrent program. It tracks every connected device, each of which the standards call a user equipment (UE), runs Radio Resource Control (RRC) signalling [4], and coordinates mobility and handovers inside strict latency limits.

A busy site runs thousands of connection-management and mobility procedures per second, each inside the millisecond-scale budget the 5G system sets for control-plane signalling [5], and may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network, so an outage or a tail-latency spike here is not a local defect: every device the site serves is affected [6, 7].

That software is written mostly in C++, which supplies the low-level control, mature tooling, and performance latency-critical systems demand at a well-known cost. Memory and concurrency management fall to the programmer, and decades of experience show the burden is routinely mishandled. The result is use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust is the first mainstream systems language to promise an escape from that trade-off, enforcing memory safety and data-race freedom at compile time on machine-checked foundations [10, 11] with performance intended to be competitive with C++ [12]. Whether the promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, develops two functionally equivalent implementations of a representative RAN subsystem, a *UE Context Management (UECM)* service modelled on a real Nokia *gNB-CU-CP* component, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload. The property that decides whether such a system meets its deadlines is *tail latency*, the p99 and p99.9 percentiles and not the mean [6, 7].

1.1 Motivation and industrial context

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN itself. Moving control-plane functions onto general-purpose servers raises rather than lowers the premium on software quality: the software is longer-lived, evolves continuously, runs at higher concurrency, and must meet its deadlines without the determinism purpose-built hardware once provided. Open-RAN disaggregation accelerates that shift, turning the base station into virtualised, multi-vendor software connected by open interfaces [3],

so the control plane a vendor ships competes on software-engineering quality (reliability, predictable timing, security, and the ease of evolving a large codebase safely) and no longer on purpose-built hardware. A latency regression or a memory-corruption defect is then a network-wide reliability and security risk and not a localised bug, and language selection becomes a concrete engineering decision.

The second trend is that artificial intelligence (AI) is moving from a tool for tuning the network into a workload the network is built to carry, and the next generation is specified that way from the start. International Mobile Telecommunications-2030 (IMT-2030), the International Telecommunication Union (ITU) framework for 6G, lists artificial intelligence and communication among its six usage scenarios and ubiquitous intelligence among its four design principles [13]. The architecture forming around that is AI-RAN, which orchestrates telecommunications and AI workloads together on shared, accelerated infrastructure, so the sites that run the radio also host distributed edge AI platforms [14]. The AI-RAN Alliance states the operational requirement directly: one common infrastructure has to run both kinds of workload concurrently, and their service-level agreements may differ [15]. Control-plane software therefore no longer has the host to itself. It contends for central processing unit (CPU), memory, and cache with inference work whose arrival pattern it does not control, and a co-tenant that takes capacity away lowers the load at which the control plane saturates. Saturation is where the two languages separate most sharply in the measurements of Chapter 4, so memory footprint and behaviour past that point are becoming selection criteria in their own right, and neither is settled by the architecture above the code.

The third trend is an industry-wide shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [8, 9, 16, 17]. The case no longer rests on vendor reporting alone: a group spanning academia and industry has argued that memory safety now needs standardised terminology and requirements if it is to be specified and procured at all [18]. National policy has followed, with cybersecurity agencies urging software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ [19, 20]. In the market this subsystem is sold into it is a regulatory matter too. The European Union (EU) Cyber Resilience Act places secure-by-design and lifecycle vulnerability-handling obligations on manufacturers of products with digital elements, with the vulnerability-reporting obligations applying from September 2026 and most of the remainder from December 2027 [21]. It names no programming language, and it would be wrong to read it as requiring one. What it changes is that the recurring defect classes of a memory-unsafe language become a compliance exposure as well as an engineering one. Rust, meanwhile, has moved from a niche language into operating-system kernels, cloud infrastructure, and increasingly telecommunications: production hypervisors such as Amazon Web Services (AWS) Firecracker, which underpins its serverless platform, are already written in Rust [22]. The evidence base for Rust's suitability, though, rests largely on general-purpose software and average-case benchmarks, so the questions a telecom vendor must actually answer, about tail latency and predictability, about which defect classes are genuinely prevented, and about the cost of incremental adoption inside a large C++ codebase, have no direct and controlled answer. Providing

one for a representative subsystem is the motivation for this work.

Industrial setting. This thesis is carried out at Nokia, one of the few vendors that build, maintain, and evolve production RAN software end to end. That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [23, 24]. Nor is the gNB-CU-CP a minor component. In the disaggregated architecture of the 3rd Generation Partnership Project (3GPP) it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle [1, 4] under the millisecond-scale procedure budgets the 5G system places on control signalling [5] and at the event rates and connected-device counts a deployed network sustains, and Section 2.1 places it in the wider architecture. Being written in C++, it inherits both that language's performance and its memory-safety and concurrency hazards, which is where the three trends converge. Software of this class is shipped, evolved, and operated across live operator networks, so the language-selection decision is a production concern rather than a hypothetical one. UECM is a deliberate choice of study object within that setting, for the reasons Section 3.1 sets out: it is modelled on a real gNB-CU-CP control-plane component rather than a textbook example, and it is small enough to reimplement faithfully and reason about in full while still carrying the properties that make control-plane code hard. Its implementation is driven entirely by a public, frozen specification rather than by proprietary code, so the comparison stays reproducible by readers outside Nokia.

A decision with a generational horizon. A language choice in software of this class is not revisited often, and it outlives the generation it was made for. The next one is being specified now: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, against an ITU calendar that asks for technology proposals in early 2029 and the submitted specifications by mid-2030 [25]. That work explicitly covers migration from and interworking with 5G, so a control plane written for the current generation is maintained alongside the next one rather than retired when it arrives. Nor do the next generation's objectives relax what motivates this comparison. IMT-2030 names security and resilience as one of its four design principles, expects the system to be secure by design, defines resilience as the ability to keep operating correctly during and after a disruptive event, and carries the ultra-reliable low-latency scenario forward in strengthened form [13]. Freedom from whole classes of defect and predictable behaviour under disturbance are being written into the next generation's objectives instead of left to implementers.

The policy dates named above fall inside the same window. The Cybersecurity and Infrastructure Security Agency (CISA) and the Federal Bureau of Investigation (FBI) classify developing new product lines in a memory-unsafe language as a bad practice where the product serves critical infrastructure, a scope mobile-network infrastructure sits squarely inside, and they exempt an existing product from the memory-safe roadmap expectation only when its announced support ends before January 2030 [26]. A vendor that defers the language question until the next generation is specified will not thereby have avoided making it. It will have made it by default, in the incumbent language, for

another decade of deployments.

What this thesis contributes to that decision is bounded. The measurements are taken on a subsystem modelled on a current-generation component and evaluated against requirements that are already published, and no claim is made about requirements not yet written. What crosses the generational boundary is not a latency figure but the language-level comparison itself: Which defect classes a compiler can rule out, what the safety model costs, and what incremental adoption inside a large C++ codebase involves. Those questions have the same answers whichever generation the specification above them names, and they are cheapest to answer before the next codebase is started.

A concrete failure mode. To make the stakes concrete, consider a mobility storm in which many UEs hand over at once. A single mishandled pointer, a UE context freed on release while an in-flight signalling message still references it, produces a use-after-free. In C++ this compiles cleanly and may pass ordinary testing, only to corrupt an unrelated UE's state or crash the worker thread under precisely the high-concurrency conditions the system exists to handle. The same latent race can equally show up as an unpredictable tail-latency spike instead of an outright crash. In safe Rust the same design does not compile: ownership and borrowing reject the dangling reference at build time. This one scenario captures why the thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload) rather than average throughput alone.

1.2 Problem statement

Two properties of this domain make the language choice consequential. Manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. And what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [6], which is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, yet vendors under growing pressure, technical and increasingly regulatory, to adopt memory-safe languages lack *direct, controlled* evidence for the RAN control plane. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [12, 27], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis solves.

1.3 Research questions and hypotheses

Four research questions organise the thesis, ordered to put predictability and migration ahead of raw average performance:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99, p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

Three prior expectations accompany them, stated in advance so the evaluation can confirm or refute them instead of describing results after the fact. **H1:** For this workload the two languages are performance-equivalent in the operationally relevant region below saturation, with any difference confined to the far tail. **H2:** Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ leaves to best-effort, after-the-fact detection. **H3:** The run-time cost of C++-to-Rust interoperability is negligible relative to the cost of the operations invoked, so that the true cost of incremental adoption is engineering effort rather than performance. H2 is stated so that it can fail: the defect catalogue behind RQ2 spans classes safe Rust is expected to prevent and classes it is expected not to, so the study can locate the boundary of the guarantee instead of only confirming it. Section 5.3 revisits all three against the evidence.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The next-generation radio access network (NG-RAN) architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE [1, 2, 4]. Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signalling storms), strict and predictable latency bounds that the 5G service and RAN specifications quantify as key performance indicators (KPIs) [5, 28], long-lived software with continuously evolving complexity, and heavily concurrent workloads. In these systems the worst-case and tail behaviour, rather than the average, determines whether deadlines are met, which motivates the predictability focus of this thesis.

Functionally, the gNB is disaggregated along two independent axes. The first is a *central/distributed* split: a central unit (gNB-CU) hosts the higher layers of the protocol stack, while one or more distributed units (gNB-DU) host the lower, more time-critical layers close to the radio. The second is a *control/user-plane* split inside the central unit, separating the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP [1, 2, 29]. Open-RAN initiatives standardise these interfaces further so that each part can be supplied by a different vendor and run as software on commodity hardware [3]. Figure 2.1 places the subsystem studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP: it owns the per-UE control-plane state and mediates the RRC signalling and mobility procedures, which is why its correctness and timing behaviour are representative of the control plane as a whole.

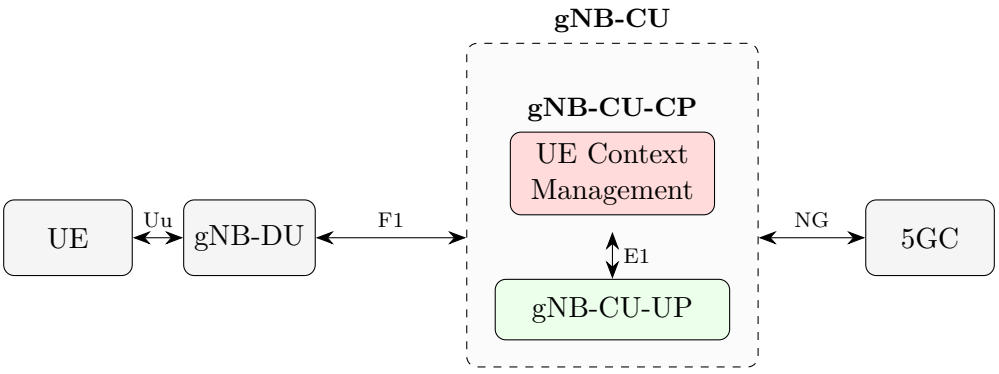


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. The gNB is split into a distributed unit (gNB-DU) and a central unit (gNB-CU), and the central unit is split again into control-plane (gNB-CU-CP) and user-plane (gNB-CU-UP) functions, connected by the standardised F1, E1, and NG interfaces. UE Context Management (highlighted) is the per-UE control-plane state manager inside the gNB-CU-CP that this thesis reimplements and measures.

2.2 Predictability and the two languages

Four strands of background bear on the comparison, and this section states each as the position the study inherits from it. Predictability, not average throughput, is the governing property of a latency-critical service, so the quantity to measure is the tail of the distribution and not its mean, and measuring it correctly constrains the harness as much as the metric: an open-loop schedule is required, because a closed-loop one stops issuing requests while a stall is in progress and so never records the latency of the stall.

C++ supplies the control the domain demands and leaves memory and concurrency safety to the programmer, with sanitisers and review standing in for a guarantee. The standards committee's work on enforced safety profiles is neither standardised nor generally implemented. Rust's ownership and borrowing discipline rules out lifetime and aliasing errors at compile time, and its `Send` and `Sync` traits rule out data races. That is a narrower claim than the phrase "thread safety" suggests. It says nothing about lock-acquisition order, about the atomicity of a composed operation, or about leaks, and it is suspended wherever a program writes `unsafe`. Both languages can express the same concurrency structures, and a comparison that lets the structure vary with the language measures the structure. The two meet through a C application binary interface, which is the mechanism any incremental migration has to use and the reason a seam concentrates whatever `unsafe` a migration needs.

2.3 Research gap

The preceding sections set up the gap. The RAN control plane is a soft-to-firm real-time, highly concurrent domain where tail behaviour under overload, not average throughput, governs the quality the network delivers. C++ supplies the control and performance the domain demands, but leaves whole classes of memory and concurrency defects to best-effort detection after the fact. Rust promises to prevent those classes at compile time at no run-time cost, and offers a credible incremental adoption path through the C application binary interface (ABI).

The domain is not unstudied, and the volume of prior work is easy to underestimate because almost none of it is indexed. At least five vendor-hosted master's theses, all carried out at Ericsson, have compared candidate languages for RAN and telecom software. One weighs runtime cost against development efficiency across C++, Rust, and Go [30]. One asks whether Rust could replace C in RAN node management [31]. One builds a thread-per-core packet processor on asynchronous Rust and measures it against the Data Plane Development Kit (DPDK) application it mirrors [32]. One measures median and 99th-percentile latency for control-plane procedures across five asynchronous programming frameworks [33]. The last builds matched C++ and Rust implementations of an Abstract Syntax Notation One (ASN.1) decompiler and compares them statistically [34]. All five are directly relevant, and two report differences unfavourable to Rust.

Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [30, 32, 33]. The effect sizes reach a factor of five, which is an indication of

how much of the measurement the runtime owns. The remaining two hold the design closer to fixed but report execution time and peak memory rather than tail behaviour under rising load [31, 34], and the closest of them, Emberg and Wadsten, measures a stateless transformation rather than a stateful, concurrent subsystem. Figure 2.2 sets the neighbouring studies against the properties a controlled answer would require, and the pattern it shows is that each holds some of them and none holds all.

The concentration of that evidence is itself part of the gap. All five studies come from one vendor, and their conclusions have never been tested against another organisation's software, targets or conventions, so what is known about this question is what one company's frameworks report about it. The published record on the other side is emptier: there is no study of Rust among the indexed publications of the vendor hosting this work, and the nearest peer-reviewed position from that side argues for making memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [35], which is the strategy this thesis tests against. The paragraphs below compare the five with this study directly.

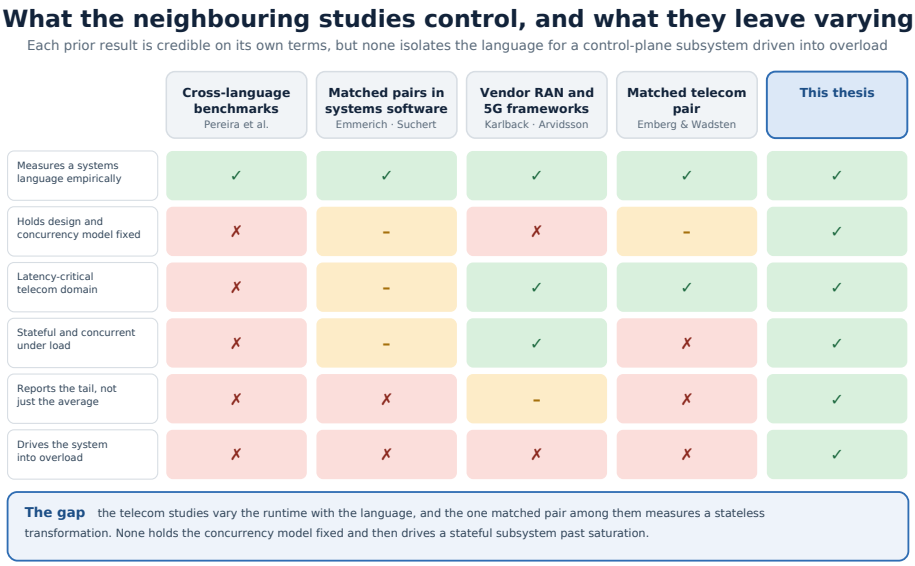


Figure 2.2: What the neighbouring studies control, and what they leave varying. Each row is a property that a controlled answer to the research questions would need. Each column is a study. The general cross-language comparisons measure the right kind of object but the wrong regime, reporting averages on general-purpose workloads. Matched pairs in systems software hold the design fixed, but outside this domain and below saturation. The vendor RAN and 5G framework studies reach the right domain, and one of them reports the tail, but in all of them the language varies together with the runtime and the concurrency model. The one matched telecom pair, Emberg and Wadsten, controls the design but measures a stateless transformation. Holding the design and the concurrency model fixed for a stateful, concurrent subsystem and then driving it past saturation is what this thesis adds. The contribution is the control, not a new measurement technique.

What is therefore missing from the literature is *direct, controlled* evidence for a representative control-plane subsystem: Whether Rust actually holds its tail latency and predictability under overload against modern C++ *when the concurrency model, algorithms, and data structures are held fixed*, exactly which defect classes it eliminates and at what stage the remainder are caught, what its safety model costs in throughput and resources, and what incremental adoption costs in practice. Closing this gap is the purpose of the

four research questions. The next chapter grounds them in the specific subsystem, defect classes, and timing properties at stake, and sets out the methodology by which they are pursued.

3 Research Methodology

The preceding chapter set out the gap this study closes. This one sets out how, beginning with the subsystem the work is carried out on and the defect classes and timing properties at stake in it. It then states the overall research strategy, the method chosen for each research question and why it is appropriate, the controls that make a language comparison fair, the statistical treatment of the measurements, and the reproducibility and ethical commitments under which the work is carried out.

3.1 The UE Context Management subsystem

The subsystem under study is a UE Context Management service, modelled on a real Nokia gNB-CU-CP service domain that owns the UE context lifecycle, RRC procedure execution [4], and mobility/handover, the functions the 3GPP architecture assigns to the control-plane part of a disaggregated gNB (Section 2.1). Figure 3.1 shows the UE-context lifecycle state machine both implementations encode identically and the mixed workload of Section 4.2 walks each UE through.

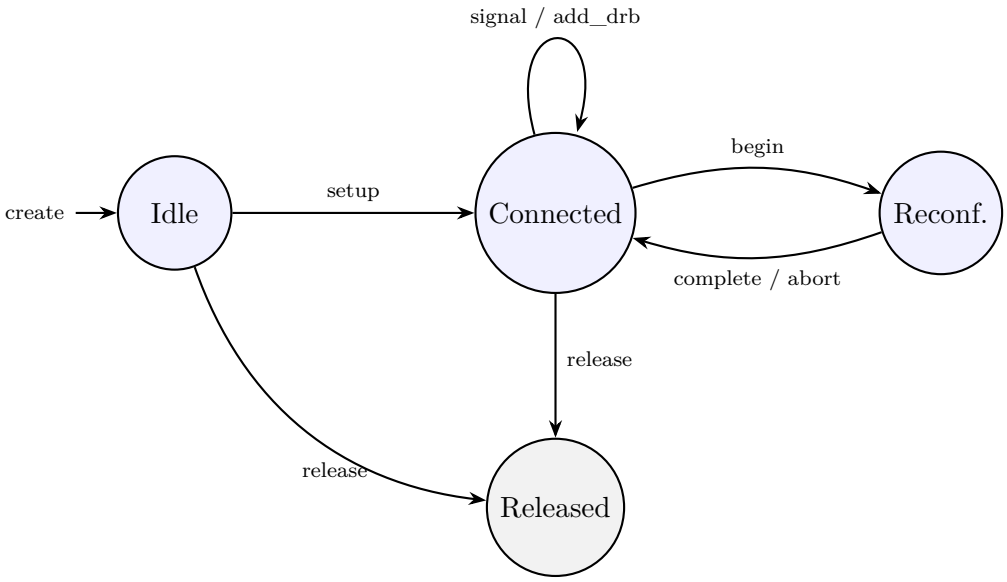


Figure 3.1: UE-context lifecycle state machine. `create` allocates a context in `Idle`, `setup` connects it, and `signal` and `add_drb` are handled in place while `Connected`. Operations invalid for the current state return an `OpError` instead of changing state.

Why this subsystem. UECM is core RAN layer-3 functionality carrying the properties that make production control-plane code difficult: per-UE mutable state, a strict operation lifecycle, high concurrency across UEs, and the latency and reliability targets the 5G system sets for control-plane procedures [5, 28]. Conclusions about how a language handles those properties therefore transfer to other stateful, concurrent control-plane services. It is also small enough to specify precisely, implement faithfully in two languages, and reason about in full, which is what makes a parity-tested comparison possible. A

larger subsystem would buy external validity at the cost of the internal validity a fair language comparison depends on.

Operation set. The interface exercised by the study is deliberately small and lifecycle-oriented: create and release a context, set up a connection (`Idle→Connected`), handle an in-session signalling message, add a data radio bearer, begin, complete, or abort a reconfiguration or handover, and look up a context's state under concurrent load. Every operation is defined for every state of Figure 3.1, and one illegal in the current state returns a typed error and leaves the context unchanged instead of throwing or corrupting state. That closed, well-typed set lets the same test vectors and the same workload drive both implementations along identical paths.

The defect classes this subsystem can exhibit. RQ2 needs a catalogue and not a list of hazards, and one is built from this operation set. It is 14 classes in two groups of seven, and the split is the falsifiable commitment: seven drawn from inside the region safe Rust claims to govern, and seven chosen precisely because safe Rust is expected *not* to prevent them, each with its expectation recorded before the programs were run. Every class is mapped to an operation that would exhibit it, so the catalogue answers to this subsystem and not to memory safety in the abstract, and Section 4.4.4 reports what each one did.

Why the tail is the metric here. Section 2.2 makes the tail the governing quantity for a latency-critical service, and two properties of this subsystem are why it governs here. One user-visible action, a call setup or a handover, becomes many dependent control-plane operations whose slowest member gates completion, and a busy site drives thousands of such procedures per second, so a percentile far out in the distribution is reached often enough to matter. The second is the shape of the failure: a service of this design tracks its offered load with flat latency until a worker saturates, and then latency rises by orders of magnitude within one increment of load, which is why this study sweeps past saturation instead of stopping at a nominal operating point.

3.2 Research approach

The study is experimental with a constructive (design-science) element. The artifact is a pair of functionally equivalent implementations of the subsystem, one in C++ and one in Rust, plus the framework that benchmarks them. Both are built from a single frozen specification and then evaluated under controlled experiments. Evidence is triangulated across three methods. Performance measurement under a load sweep answers RQ1 and RQ3, a controlled fault-injection study answers RQ2, and an FFI migration case study, part measured and part qualitative, answers RQ4. The three are independent. A weakness in one of them, such as the environment sensitivity of tail-latency measurement, therefore does not carry into the conclusions as a whole. Every measurement reports variance over repeated runs, and Section 3.4 sets out the controls that keep the language the only variable.

That design follows from the shape of the questions. Each asks how one outcome (tail latency, defects prevented, resource use, migration cost) depends on one variable, the implementation language. The language therefore has to be the only systematic difference between the two objects under test. This rules out the commonest form of language comparison in the literature, where two independently written programs are measured and the gap is attributed to the language. Such a gap also carries the design, the libraries and the skill of their authors.

Building both sides from one specification removes that confound. The comparison is then between two encodings of the same design, and the artifact becomes both the contribution and the instrument that measures it. Where a question can be answered with numbers, the treatment is quantitative. Where it cannot, as with the kind of guarantee in RQ2 and the application programming interface (API) design effort in RQ4, it is qualitative but still systematic.

The gap this design addresses (Section 2.3) is only as good as the search that looked for it, so that search is on record. Appendix B gives the databases, the term combinations, the citation chaining and the filtering rules. Two of its findings bear on the design and not just on the bibliography.

The vendor-hosted theses closest to this study appear in no citation database. Chaining therefore never reaches them, and they had to be found by searching the institutional repositories directly. The same searches establish an absence on the other side: no Finnish thesis on Rust in a radio-access or telecom context, and no indexed publication mentioning Rust from the vendor hosting this work. The domain-specific literature on the language question is therefore Swedish and Ericsson-hosted, and whether it carries over here is untested. Those theses are grey literature, so they are cited as evidence of what a vendor has attempted and found, not as peer-reviewed results.

The search returned no study, indexed or grey, that measures the tail-latency behaviour of a Rust and a C++ implementation of the same stateful, concurrent control-plane subsystem under overload.

Table 3.1 maps each research question to the method, artifact, data and metric that answer it. All four chains end in the same place: Section 5.2 judges each question and Section 5.3 disposes of its hypothesis. RQ1 and RQ3 share H1, worded to cover performance equivalence in both the latency and the resource sense. That wording turns out to be the hypothesis's main weakness. RQ2 is insensitive to the environment, and RQ4 is so except along one axis: build time is a timing measurement like any other, and is therefore read as a ratio between two toolchains measured alternately on one machine. RQ1 and RQ3 depend on the measurement host and are reported against what it can carry (Section 4.1).

Table 3.1: Research design: how each research question is answered. Every question is tied to a concrete method, a published artifact, the data it produces, and the metric on which the conclusion rests. RQ1 and RQ3 carry two methods rather than one: the sweep measures the round-trip a deployment would experience, and the component decomposition measures the part of it the language is responsible for. Section 3.3 states why one cannot stand in for the other. RQ1 and RQ3 are tested against H1, RQ2 against H2, and RQ4 against H3. The RQ2 row abbreviates the three dynamic sanitisers the C++ side is built under: AddressSanitizer (ASan), ThreadSanitizer (TSan) and UndefinedBehaviorSanitizer (UBSan).

RQ	Method	Artifact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99 tail latency, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak resident set size (RSS), CPU utilisation, throughput
RQ1, RQ3	Component decomposition: the core called directly, then through the service	Matched microbenchmark pair, interleaved repeats	per-operation cost split into the operation and the cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, ASan/TSan/UBSan, `rustc`, Loom/Shuttle and Miri logs	defect-class taxonomy (compile-time / runtime / undetected)
RQ4	FFI migration prototype	C++ host + Rust C-ABI seam	seam size, `unsafe` scope, per-call interop cost

3.3 Methods per research question, in outline

RQ1 and RQ3 are answered by an open-loop load sweep from light load to well past saturation, with the four build variants interleaved inside every repeat, and by a second measurement that separates the operation from the cross-thread handoff around it, because the end-to-end round-trip bundles two costs of different character and only one of them can differ between the languages. A third isolates the contended queue and completion primitives on their own, so the overload attribution can be tested and not merely argued.

RQ2 is answered by controlled fault injection. Each defect class is written as a small self-contained program in both languages, the C++ side built and run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer, and the safe-Rust side compiled and, where it compiles, run under a bounded timeout. No static analyser is run, deliberately, so the C++ columns report the tooling this study ran and not a ceiling on what C++ tooling could find. RQ4 is answered by a working migration prototype in the direction a real migration takes, a C++ host calling an unmodified Rust core across a C application binary interface, with the per-call cost measured on a real operation and not inferred from an empty crossing.

3.4 Fairness as a controlled variable

A language comparison is only as good as the variables it holds fixed. The specification is frozen before either implementation exists. The concurrency model, the allocator and its arena count, the optimisation level, the hash function, the histogram algorithm and the load schedule are matched across the two sides, never tuned per side. Where the two languages could not be written alike the divergence is logged instead of resolved, and behavioural equivalence is established by diffing conformance transcripts operation by operation. One row of that table has no control against it: one person wrote both implementations and is not equally expert in both, which Section 5.5 treats as the study's principal internal threat.

3.5 Statistical treatment

Three commitments of the statistical design are fixed in advance, because the study's credibility rests on their having been fixed before any data was seen. The family of tested comparisons is enumerated ahead of the campaign, so an operating point identified after seeing a sweep cannot enter it however small its p-value would be. Each comparison is an exact paired permutation test over 20 interleaved repeats with a bootstrap confidence interval, Holm-corrected across the family. The attainable floors (0.0000019 uncorrected, 0.000027 corrected) are quoted so that a null can be distinguished from an arithmetic limit. And two quantities are deliberately excluded from the family: inter-operation jitter, which no research question asks for, and the pooled p99.99 far tail, whose unit of analysis is the operation and not the run, so it has no across-repeat interval to test.

3.6 Reproducibility and research artifacts

The study is designed so that its results can be regenerated by a third party without access to Nokia's proprietary code. Three things make that possible. The implementations are driven entirely by the public frozen specification. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published together with the thesis. And every result table, every plot of measured data, and every quoted number in the evaluation is rebuilt by one regeneration script in the published package (`bench/refresh_thesis.sh`) directly from the raw logs. The scope of that claim is worth stating precisely, because two things it might be read to cover are outside it. The hand-drawn schematics that explain a concept rather than report a result are authored and not generated. And running the same scripts over another host's campaign regenerates the artifacts by the same procedure and not with the same values. What the script guarantees is that no figure, table, or sentence can hold a value the logs no longer support.

The logs that script reads are part of the package. A performance campaign is an append-only newline-delimited JavaScript Object Notation (JSON) file, named by the timestamp at which it started. The reported sweep and the virtualised campaign it replaced both sit under `bench/results/`, while the second bare-metal campaign has a directory of its own. Each campaign has its own script and its own macro prefix, so a figure from one cannot reach a sentence about another. The two bare-metal campaigns also write a side-

car holding every run's complete histogram buckets, which is what the far-tail percentile is computed from and which the virtualised campaign predates. Each run's environment is recorded by `bench/capture_env.sh` and stored beside the campaign data. Three runners write three machine-readable outcome files under `faults/results/`: `taxonomy.csv`, `model_checking.csv`, and `miri.csv`. Nothing in the evaluation chapters is transcribed from those files by hand. The execution environment travels with its data, which is what lets the thesis separate environment-sensitive results from environment-insensitive ones. Nothing on the path requires proprietary source.

3.7 The two implementations in brief

The properties of the design and of both implementations that the evaluation depends on are these. Both are built from one frozen specification, written before either implementation existed, which fixes the operation set, the state machine and the error model before either version could negotiate them. Both use the same concurrency structure, a shared-nothing thread-per-shard pool in which each worker owns its slice of the UE state outright, reached through a bounded blocking queue of fixed capacity, so no lock protects shared mutable state on the hot path in either language. A request crosses that queue as a tagged value carrying a one-shot completion handle, which is what makes the end-to-end latency a submit-and-complete round-trip and not a function call.

Where the two could not be written the same way, the divergence is logged and left unresolved in either direction. Two of those divergences carry the overload result of Section 4.4.2: the bounded queue, hand-written in C++ against Rust's `sync_channel`, and the completion handle, `std::promise` against a fresh `mpsc` channel per request. A third carries the operation-level result of Section 4.4.1, since the per-shard container is each language's own standard hash map and those are different data structures. The hash function itself is pinned to the Fowler–Noll–Vo hash function (FNV-1a) on both sides precisely so that hashing cannot be the explanation. Behavioural equivalence is not assumed either: a language-neutral conformance suite drives both through 223 hand-written and 793 000 generated operations and diffs the canonical transcripts operation by operation.

4 Experimental Setup and Evaluation

4.1 Testbed configuration

This chapter reports two kinds of result. Some follow from the compilers and the source, others depend on the machine. Table E.1 describes the machines and keeps that split clear.

Every performance number here comes from one dedicated bare-metal server: a single-socket AMD EPYC 7443 with 24 cores on 1 non-uniform memory access (NUMA) node, simultaneous multithreading (SMT) off, kernel 6.17.0-23-generic. The frequency governor is fixed at performance with boost disabled, so the cores hold their base clock, and cores 2–17 carry the benchmark. The environment capture records virtualisation as none, and every run records the metadata of Section 3.6.

The toolchain is not the host's. Both implementations are built and run inside a pinned container image carrying Rocky Linux 8.10 (Green Obsidian) with `g++ 15.2.1` and `rustc 1.97.1`, the same image and the same `rustc` the earlier virtualised campaign used. The image pins the `gcc-toolset` major version and not the release within it, so that campaign compiled with `g++ 15.1.1`, one GCC 15 release earlier. The container also takes the kernel from its host. The two campaigns therefore differ in the hardware, in the kernel (6.17.0-23-generic against 7.0.2-1.el8.elrepo.x86_64), and in that compiler release.

Two properties of this host are what the previous one lacked. No hypervisor sits between the benchmark and the cores, so no timeslice is lost to a machine this study cannot observe, and the kernel runs tickless on the pinned cores (`nohz_full`), which removes the periodic timer interrupt from the measured path. What the host does *not* have is `isolcpus`, deliberately: isolating cores also removes them from every scheduling domain, and because the benchmark pins its threads to a range and relies on the kernel to spread them, an isolated set collapses all of them onto one core. Section 5.5 records the cost.

Until late in the work the campaign ran on a virtualised host instead: a Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs, kvm virtualisation, no core isolation. Three of this chapter's results were already settled there and one was not settleable there at all, and the obstacle was not a shortage of repeats. On that host the 95% confidence interval on a single variant's p99 spans 1480 μs while the four variants' p99 values differ by 293 μs, so the uncertainty about one swamped the gap between all of them. That campaign is not discarded. It shares this one's design exactly, the same 7 shards and 7 producers over 14 load levels at 20 repeats, so the two can be set side by side to ask which findings belong to the code. That comparison is the only direct evidence this study can offer that its instrument is adequate to the questions it asks, and one of its findings is that a quantity this thesis once reported as a property of allocators was in part a property of that machine.

On either host Section 3.5 applies: each sweep point is a median over 20 interleaved repeats with a bootstrap 95% confidence interval. Interleaving shares interference across variants, which costs absolute latency and preserves the comparison. Throughput is counted over the measured phase only, warm-up excluded.

4.2 Workload and evaluation measurements

A synthetic but spec-justified telecom workload is generated by a deterministic driver whose structure both implementations share, so the two versions are exercised along identical code paths. The *signal* workload preloads a population of connected UEs and hammers the `handle_signalling` hot path, isolating steady-state per-message latency. The *mixed* workload walks each UE through a realistic procedure sequence, context creation, connection setup, signalling, bearer setup, reconfiguration or handover (begin/complete/abort), and release, using a fixed SplitMix64 draw order and a per-slot expected-state model so that every operation it emits is valid. The fixed draw order makes the per-operation-kind counts identical in C++ and Rust for a given active-UE population and seed, which doubles as the parity signal of Section 3.7. The control parameters are the target event rate and the number of active UEs, swept from nominal load through saturation and overload. The send schedule is fixed and the histogram is corrected, so a stall introduces no coordinated-omission bias [36, 37]. Three campaigns are run: a load sweep for performance, the fault-injection study, and an FFI migration prototype. Figure 4.1 shows the two workloads and the sequence the mixed one walks.

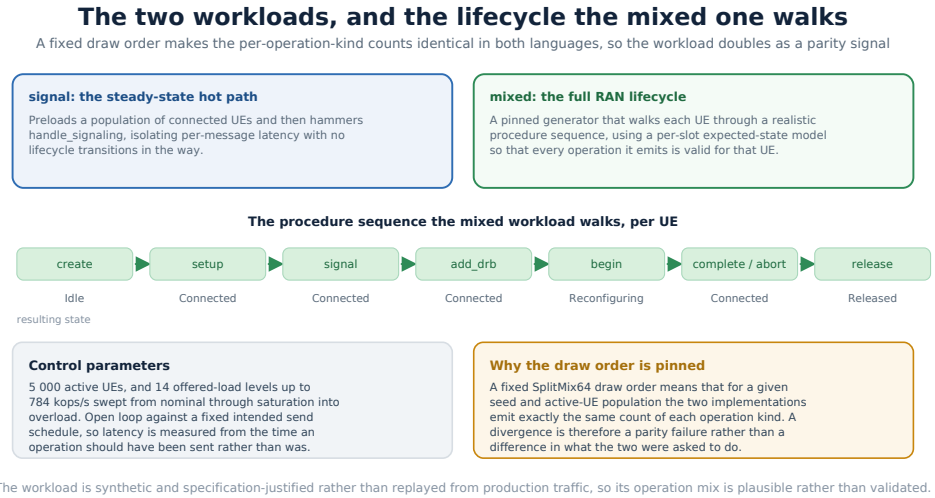


Figure 4.1: The two workloads. The *signal* workload isolates the steady-state hot path. The *mixed* workload walks each UE through the full RAN procedure sequence, with a per-slot expected-state model ensuring that every operation it emits is legal for the state its context is actually in. Because the SplitMix64 draw order is pinned, a given seed and active-UE population produce identical per-operation-kind counts in both languages, so a divergence is a parity failure rather than a difference in what the two implementations were asked to do.

The term *spec-justified* is used precisely, because the distinction bounds what the workload can support. The *sequence* of operations is justified. It is the RRC connection lifecycle the 3GPP specifications define for a UE, from establishment through in-session signalling and bearer setup to reconfiguration and release [1, 4], and the per-slot expected-state

model guarantees that every operation the driver emits is legal for the state its context is in. The *proportions* are not justified in the same way. How often a real cell hands over, against how often it establishes a bearer, depends on deployment, mobility and time of day, and no public figure fixes it, so the mix used here is plausible and not validated. Section 5.5 reports that as a threat to external validity and trace replay heads the future-work list for it. For a *comparative* question it costs less than it would for an absolute one: both implementations execute the same sequence in the same order for a given seed and active-UE population, verified by the per-operation-kind counts, so an unrepresentative mix shifts both languages equally and cannot manufacture a difference between them. It bounds how far the numbers travel and not the fairness of the comparison.

Table E.2 gathers the configuration of the service, the workload and the campaign in one place, since those values are otherwise spread across the specification, Section 3.7 and this chapter. Every entry in the campaign block is either fixed by the specification or generated from the measurement logs. Two operating points are used throughout, and the first of them is fixed in advance, not derived from the data. The representative point sits at 56 kops/s against a measured throughput plateau of 392 kops/s, and it is fixed because the alternative was tried and failed. The rule this study began with took the busiest swept load at which all four variants still held their p99 below 5 ms. It is retained as a diagnostic and is no longer the operating point, because it measures the machine and not the service. On the virtualised host it returned 56 kops/s, where p99 sat just under the deadline, so it was locating where that host's interference reached 5 ms. On a quiet host p99 stays near 74 μs until capacity binds, so the same rule climbs until it collapses the representative point onto the saturation knee. Fixing the point costs the appearance of derivation and buys two things worth more: the hosts are compared at the same offered load, and the point stays below saturation on all of them, which every below-saturation claim in this chapter assumes.

4.3 Evaluation data

The campaigns behind this chapter differ in kind: one load sweep, three small paired experiments, a categorical fault catalogue, a build and footprint measurement, and two further sweeps kept for comparison. Appendix E records what each captured, field by field, because judging the analysis takes knowing the record and not only the tables drawn from it. Every measured quantity this chapter quotes is a generated macro (Section 3.6). The software-quality proxies sit beside the forced divergences they explain and beside the FFI seam, the only place either implementation contains `unsafe` (Table 4.10).

4.4 Evaluation results

The safety and most of the migration results do not depend on the measurement host. The performance and resource results do, and every figure and table below is measured on the host of Section 4.1 unless stated otherwise. Every comparison on that host resolves, including the far tail below saturation. The divergence past saturation is resolved as a measurement and not as a mechanism, and Section 4.4.2 reports how far the mechanism could be narrowed.

4.4.1 Below saturation (RQ1, RQ3)

The hypothesis under test is H1 (Section 1.3): equivalence below saturation, with any difference confined to the far tail. Each point aggregates 20 interleaved repeats, and reported values and shaded bands are medians with nonparametric bootstrap 95% confidence intervals. Table 4.1 summarises per-operation latency at the representative load of Section 4.2, 56 kops/s, a point at which all four build variants are simultaneously healthy.

Two readings of that table answer different questions. Magnitude is negligible: the four variants sit within 2 μs of one another at the median, 2.1%, which no deployment would notice. *Ordering* is legible: the two Rust intervals sit below the two C++ intervals with no overlap between the languages, while the two allocators of one language overlap each other. On the previous host every interval at this percentile overlapped every other and the ordering had to come from the test alone. That agreement extends up the distribution. The p99 values differ by 2 μs across variants while the widest 95% confidence interval on a single variant's own p99 spans 1 μs, so the uncertainty about one variant is smaller than the gap between them, which is what licenses an ordering. It is also the relation this study could not obtain for most of its life: on the virtualised host the same two quantities were 293 μs and 1480 μs, the wrong way round by a factor of five, which is a result about measurement rather than about either language.

The tail rows of Table 4.2 therefore carry a finding, and the instrument that carries it is the corrected p-value and not the interval. Both p99 rows and both p99.9 rows clear the Holm correction, 4 rows in all, in the same direction as at the median. Their intervals also exclude zero, and Section 3.5 fixes in advance that an interval in this block is not read as a finding, so those are reported and left to one side. A rule dropped once the machine improves was never a rule.

Predictability is evidenced as far as p99.99, and that column is a different statistic from the rest of its table: one percentile of the 20 repeats' histograms concatenated, between 3 360 051 and 3 360 084 operations per variant, which is what Section 3.5 means by pooling. A single run holds too few observations past p99.99 for the percentile to mean anything, and every run persists its full histogram buckets so the repeats can be combined afterwards. Pooling takes the operation as the unit of analysis, so the column has no across-repeat interval and none is printed. Read that way the far tail separates the languages outright: the widest gap between variants is 87 μs, 97% of the smallest, and both Rust variants sit below both of the others.

Table 4.1: Per-operation latency at a sustained offered load of 56 kops/s (mixed RAN-lifecycle workload, 7 shards). Values in microseconds: the median over 20 repeats above its nonparametric bootstrap 95% confidence interval, printed in full rather than as a symmetric half-width because it is strongly asymmetric at these percentiles and a half-width covering the upper bound would place the lower one below zero. †The p99.99 column is the deliberate exception, one percentile of the 20 repeats' histograms concatenated, 3 360 051 to 3 360 084 operations per row, because a per-run p99.99 rests on too few observations to read. Pooling makes the operation the unit of analysis, so that column carries no across-repeat interval. With the maximum it is one of the two columns the paired family does not test, for the same reason: both are read off exactly the operations interference lands on, which is the class pairing cannot rescue.

Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.7	76.4	81.2	142.0	243.1
	[73.5, 73.7]	[76.2, 77.0]	[80.8, 81.6]		[239.2, 275.8]
C++ (jemalloc)	73.5	76.6	80.2	176.4	246.3
	[73.3, 73.6]	[76.4, 76.9]	[79.5, 82.0]		[228.8, 259.5]
Rust (system alloc)	72.4	75.0	77.3	92.1	145.2
	[72.3, 72.5]	[74.7, 75.2]	[77.0, 78.7]		[115.9, 178.2]
Rust (jemalloc)	72.1	75.6	77.8	89.5	121.4
	[72.1, 72.3]	[75.6, 75.7]	[77.7, 77.9]		[101.8, 179.6]

Those percentiles describe the distribution and not how far consecutive operations differ from each other, which Request for Comments (RFC) 3393 names inter-operation delay variation [38]. Table E.3 in Appendix E reports it descriptively, because no research question asks for it. Median jitter across the four variants runs 1.37 to 1.72 μs, 2% of the median latency it perturbs, and what separates the four variants is smaller than what separates one load level from the next, the conclusion the second bare-metal campaign reaches independently.

Overlapping intervals are a weak instrument in both directions, so the comparison is also tested directly, by the procedure fixed in Section 3.5 and reported in Table 4.2. What licenses the pairing is a property of this campaign rather than of the statistic: the four variants are interleaved inside every repeat, so the i-th run of each met the same ambient conditions and whatever interference that block carried cancels in their difference. That is the second thing the interleaving discipline buys, beyond the fairness argument it was adopted for.

Figure 4.2 draws the same family on one axis. All 14 of the 14 tests survive Holm, none changes sign when the allocator changes, the tail intervals are the widest of the set, and the smallest adjusted value sits at the family's own floor of 0.000027. A family in which nothing fails invites the objection that the threshold is doing no work: peak resident set size under `jemalloc` did not clear it on the virtualised host, and neither did any tail row there. 14 paired intervals exclude zero across both blocks, so the reading rule of Section 3.5 changes no verdict, and an earlier seven-repeat pass produced tail intervals excluding zero that more repeats removed. The allocator control runs every comparison twice, and here p50 latency, p99 latency, p99.9 latency, Peak RSS and CPU utilisation keep their sign and 0 reverse. Section 5.5 records the campaign on which the control did veto.

Resolving a difference is not the same as sizing it, and Table 4.3 is where the size goes. Every row is significant here, so the verdict column carries no information and the margins carry all of it: at the median the effect could be no larger than 1.3 μs, and in the tail

Table 4.2: Exact paired permutation test of C++ against Rust at 56 kops/s, with the allocator held fixed. The effect is the median of the 20 paired differences (Rust − C++) above its bootstrap 95% confidence interval, the test statistic is their mean, p is exact over all 2^{20} sign assignments, and the final column applies the Holm correction across the whole family of 14 tests. The attainable floors are 0.0000019 uncorrected and 0.000027 after correction. The rows are blocked by the criterion of Section 3.5: pairing rescues the statistics in the upper block, so a consistent interval there is evidence, whereas the high percentiles below are computed from exactly the operations interference falls on, so an interval excluding zero is not read as a finding. Both blocks clear the correction here, so the rule costs the lower block nothing and is applied anyway. Achieved throughput is not tested, for the reason given in the same section, and appears in Table 4.8 instead.

Metric	Allocator	Effect (Rust − C++)	p	Holm
Robust to interference on single operations				
p50 latency (µs)	system	−1.3 [−1.3, −1.2]	0.0000019	0.000027
p50 latency (µs)	jemalloc	−1.2 [−1.3, −1.2]	0.0000019	0.000027
Peak RSS (MiB)	system	+1.4 [+1.2, +1.7]	0.0000019	0.000027
Peak RSS (MiB)	jemalloc	+2.5 [+2.4, +2.6]	0.0000019	0.000027
CPU utilisation (%)	system	−5.1 [−6.7, −4.7]	0.0000019	0.000027
CPU utilisation (%)	jemalloc	−4.6 [−5.9, −3.9]	0.0000019	0.000027
Determined by it: high percentiles				
p99 latency (µs)	system	−1 [−2, −1]	0.0000057	0.000027
p99 latency (µs)	jemalloc	−1 [−1, −1]	0.0000038	0.000027
p99.9 latency (µs)	system	−4 [−5, −2]	0.00016	0.00016
p99.9 latency (µs)	jemalloc	−2 [−4, −2]	0.0000095	0.000027

block no larger than 5 μs, both below anything a procedure budget of a few milliseconds would notice. The differences are real, reproducible and operationally negligible. That table was built to report the opposite case, and on the virtualised host it did, with 5 of its 10 rows unresolved and a tail margin running out to 1298 μs.

The median itself is a resolved difference and not a null: 1.3 μs in Rust's favour, consistent under both allocators. The interval on it reaches 1.3 μs against a median of 70 μs, and the temptation is to report that as a bound on the language effect. That reading would be wrong, for the reason set out next.

What the end-to-end number is made of. Every latency the sweep reports is an end-to-end round-trip, and Section 3.3 set out why that bundles two costs of quite different character: the operation itself, executed inside a single thread and generated by the compiler, and the cross-thread submit-and-complete handoff around it, charged to the operating system. Only the first can plausibly differ between the languages, the sweep cannot separate the two, and a bound says only as much as the quantity it is a fraction of. Both costs are therefore measured directly by the two-level method described there, in both languages and at the sweep's own shard count and context population: `ShardCore`

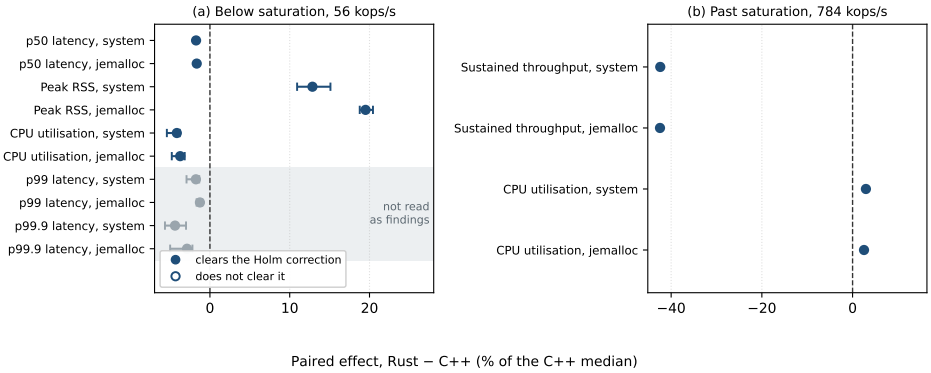


Figure 4.2: Every test in Tables 4.2 and 4.6 as a paired effect with its bootstrap 95% interval, normalised by the C++ median so that microseconds, mebibytes, percentage points and kops/s can share an axis. Filled markers are the 14 of 14 tests that clear the Holm correction, which here is all of them. The two allocators of a metric are drawn adjacent, so an effect that reversed sign between them would appear as a pair straddling the zero line. The shaded rows are the high percentiles, which Section 3.5 fixes in advance as rows whose *intervals* are not read as findings. Their corrected p-values are read, so the shading marks the instrument that is set aside and not the rows.

Table 4.3: How large each effect could be. For every comparison at 56 kops/s, the observed effect against the tightest margin within which the two variants could be called equivalent, and the verdict Table 4.2 reaches. The two questions are kept in separate tables on purpose: one asks whether a difference is there, this one asks how large it could be, and a row can matter in one and not the other. Every row resolves on the host reported here, so the last column reads the same throughout and the margins carry the information. On the virtualised host this study measured on first, 5 of these rows did not resolve, which is what the table was built to report. Latency margins are in microseconds, peak resident set size in mebibytes, CPU utilisation in percentage points.

Metric	Allocator	|Effect|	Equivalent within	Verdict
p50 latency (µs)	system	1.3	1.3	significant
p50 latency (µs)	jemalloc	1.2	1.3	significant
p99 latency (µs)	system	1	2	significant
p99 latency (µs)	jemalloc	1	1	significant
p99.9 latency (µs)	system	4	5	significant
p99.9 latency (µs)	jemalloc	2	4	significant
Peak RSS (MiB)	system	1.4	1.6	significant
Peak RSS (MiB)	jemalloc	2.5	2.6	significant
CPU utilisation (%)	system	5.1	6.6	significant
CPU utilisation (%)	jemalloc	4.6	5.7	significant

with no queue in the path gives the operation, `Service` gives the operation plus the handoff, and their difference is the handoff. Table 4.4 reframes the whole performance comparison.

The operation is 34.5 ns in C++ and 21.3 ns in Rust, and the handoff 13.58 μs against 13.59 μs. The work the languages actually perform is therefore at most 0.250% of what the sweep measures, and the complementary 99.750% of every latency in Table 4.1 is the operating system moving a message between two threads. That shape is not peculiar to this system or this pair of languages. On a shared-nothing database partitioned one per worker thread like this service, Zhou et al. report that once the per-operation work is optimised what dominates the rest is communication across isolation boundaries and not the operation [39].

Table 4.4: Where an end-to-end operation's cost actually goes, measured at 7 shards and 5 000 active UEs with a single uncontended caller. Each row is the median over 15 interleaved repeats above its bootstrap 95% confidence interval, in the unit its own magnitude calls for: the two terms differ by more than two orders of magnitude, which is the point of the table. p is the exact two-sided paired permutation test of C++ against Rust on the repeat-wise differences. The handoff is obtained as the difference between the two measured rows, so it carries the uncertainty of both. It predates the move to bare metal and carries no environment record, so the absolute nanoseconds name no host. Both arms ran alternately in one session, which is what the share and the ratio rest on.

Component	C++	Rust	p
Operation alone, no queue in the path (ns)	34.5	21.3	0.000061
	[32.4, 35.2]	[21.0, 21.5]	
Cross-thread submit-and-complete handoff (µs)	13.58	13.59	0.439
	[13.38, 13.83]	[12.69, 14.52]	
End-to-end round-trip, uncontended (µs)	13.62	13.61	0.421
	[13.41, 13.87]	[12.71, 14.54]	
Operation's share of the round-trip	0.250 %	0.160 %	

Two conclusions follow, and they pull in opposite directions. The first is that the sweep cannot bound the language effect. An interval of 1.3 μs on the paired median difference is 38 times the operation's own cost, so a language effect could be several hundred percent of the work being done and still sit comfortably inside it. What the sweep does establish is weaker and still useful: neither language disturbs the scheduler-dominated round-trip, which is what a deployment of this shape would experience. That limit comes from how the measurement is built and not from the host, so unlike the tail result moving to bare metal does not fix it. Measuring the component on its own does.

The second is that once the component *is* measured on its own, the languages are not equivalent. Rust runs the bare operation about 1.6× faster, the difference shows up in the great majority of repeats, and the paired test puts its p nearly three orders of magnitude below the conventional threshold. This decomposition is a separate experiment from the sweep and no part of its Holm family. It runs 15 repeats instead of 20, so its own attainable floor is 0.000061 rather than the 0.000027 of Section 3.5, and no correction is applied to it, as with the interoperability comparison of Section 4.4.5. The sweep does resolve language differences in *time* of its own, at the median and at both tested tail percentiles, but each is a difference in a quantity that is 99.750% handoff. Figure 4.3 draws both conclusions, and its second panel is the argument in one image: the instrument sits entirely above the quantity it would have to constrain.

The attribution needs care, because the obvious reading is the wrong one. Nothing here implicates the borrow checker, bounds checking or ownership. The candidate is already recorded in the forced divergences: the per-shard container is each language's standard hash map, and those are different data structures. C++'s `std::unordered_map` has to be node-based to keep its iterator- and reference-stability guarantees, so a lookup chases a pointer out of the bucket array into a separately allocated node, while Rust's `HashMap` is an open-addressed table that keeps entries inline and usually resolves a lookup within one cache line. The hash function is pinned to FNV-1a on both sides precisely so that hashing could not be the explanation. That is a real and reportable difference between the two languages as they are actually used, because a developer reaching for the standard hash map gets whichever one their language ships, and it is not evidence about Rust's

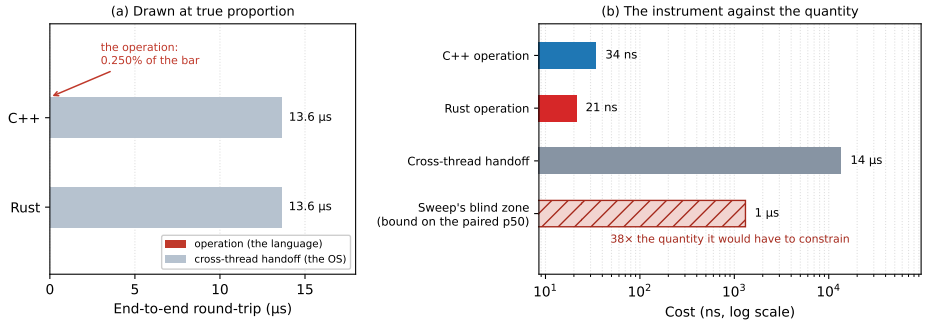


Figure 4.3: Where an end-to-end operation's cost goes, and why the sweep cannot bound a language effect. **(a)** The round-trip at true proportion: the operation the language performs is 0.250% of the bar, the remainder the cross-thread handoff. **(b)** The same quantities on a log axis against the smallest paired difference the sweep can resolve, shown as a blind zone. Both operation measurements fall inside it by a factor of 38, which is why the differences the sweep does resolve in time are differences in the round-trip and not in the work the language performs.

safety model. A second bare-metal campaign replaces the structural argument with a measurement, and its counters qualify it: the Rust build retires fewer instructions in fewer cycles while missing cache more often per thousand instructions, so what they support is less work per operation rather than the better locality this paragraph would predict. Two limits keep the finding the size it is. It is a single-threaded, uncontended microbenchmark, so it says what the operation costs and not what a loaded system does with it, and it is one operation on one data structure. Generalising from it to "Rust is 1.6× faster" would repeat the error this thesis criticises in the average-case literature.

Latency, throughput and the knee across the sweep. Figure 4.4(a) plots p99 latency against offered load, and the shape is the saturation cliff Section 3.1 described, with one feature sharper. Through the whole tracking region p99 is *flat*: 78 μs at the lightest load and 74 μs at the top of the region, a drift of -4 μs across a fifty-fold increase in offered load. A queueing model would predict a gentle climb, and there is none to see, because until a worker saturates its shard the queue never holds more than a request or two and latency is service time and not waiting time. Then, in the single step of the rate ladder that follows, p99 rises by four orders of magnitude, from 74 μs to seconds. That is the overload cliff, and how abruptly it arrives is the finding: one increment of offered load separates a service meeting millisecond deadlines from one missing them by three orders of magnitude, a design with bounded blocking queues has no graceful degradation to offer in between, and the flatness before the cliff is why, since no rising-latency region warns an operator that capacity is close. Section 3.5 tests no percentile past the knee, for two reasons. Latency there is accumulated schedule backlog and not service time, and the measured phase is cut off at a fixed width, so every reported percentile above the knee is a lower bound. What stays interpretable is how much work each variant still completes (Section 4.4.2).

Panel (b) shows achieved throughput against the same axis, and it is the more informative of the two, because an average over a whole run is far less sensitive than a percentile to interference landing on individual operations. All four variants track the ideal offered-equals-achieved line up to an offered 392 kops/s, and the lowest plateau any of them

reaches is 392 kops/s. Each knee is bracketed by the rate ladder, not pinned by it, since the next rung is where an arm stops tracking. Achieved throughput matches offered load below the knee to within 0.0%, so the operating point of Table 4.1 is a genuinely *sustained* load and its latencies are not those of an already-queueing system.

The knee does *not* sit at the same place for all four variants, which the identical thread-per-shard structure predicts that it should. The two C++ arms stop meeting their offered rate after 392 kops/s and the two Rust arms carry one rung further, to 448, a separation by language and not by allocator as it is everywhere else in this regime. At the rung between the two limits, 448 kops/s, the consequence is the largest throughput gap anywhere in the sweep: Rust still serves 448 kops/s while C++ has collapsed to 162, a factor of 2.8, in 40/40 paired repeats. Three things bound what that rung licenses. It is *not* added to the tested family of Section 3.5, because the operating point was identified after seeing the sweep and the family is fixed in advance precisely so that a point chosen that way cannot enter it, whatever its p-value would be. It is reported descriptively instead, with its per-repeat agreement, and the tested comparison remains the pre-committed overload point of Section 4.4.2. One rung of a geometric ladder is also coarse: the true C++ limit lies between 392 and 448 kops/s, so the difference in *capacity* is bounded by the ladder's spacing and not measured to better than that. And the direction here is the opposite of the direction deeper into overload, where C++ sustains more (Section 4.4.2). Both are real, they belong to different loads, and neither cancels the other.

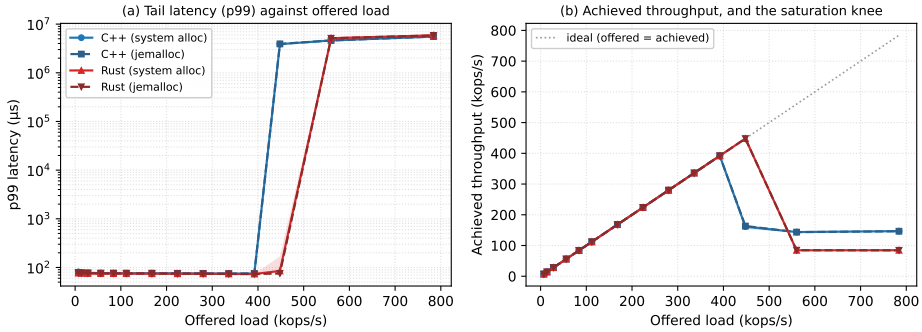


Figure 4.4: The load sweep, both panels against offered load. **(a)** Tail latency (p99) for the four build variants on a log axis, flat across the whole tracking region and then rising by four orders of magnitude in one step of the ladder. **(b)** Achieved throughput, with the dotted line marking the ideal offered-equals-achieved regime. The C++ curves leave that line after an offered 392 kops/s and the Rust curves after 448, so 448 kops/s is the only load in the sweep at which the two languages differ in *throughput* with Rust ahead. Lines are medians over 20 repeats and shaded bands are bootstrap 95% confidence intervals in both panels, which share the variant styling of the legend in (a). Throughput is counted over the measured phase only.

Figure 4.5 holds the load fixed instead and sweeps the percentile. The four variants are close at the median and fan out as the percentile rises, the expected shape for a heavy-tailed distribution and the reason a percentile-aware methodology is used throughout, and here the fan opens by language: the two Rust curves stay below the two C++ curves across the whole range, with the gap widening. Whether that is a language effect or a measurement artefact is what the right-hand panel is built to answer, and on this host the answer is the first. It sets the spread between the four variants against the widest interval on any one of them, and the interval stays the smaller quantity at every percentile

it covers, reaching at most 0.6 times the spread at p99.9, so the fan marks where the languages differ and not where the measurement runs out. That panel was added when the answer went the other way and is retained now that it does not, because a reader has no way to tell a resolved comparison from an unresolved one unless the study shows the same diagnostic in both cases. It stops at p99.9, since the far tail is reported pooled and a pooled percentile carries no across-repeat interval to weigh a spread against.

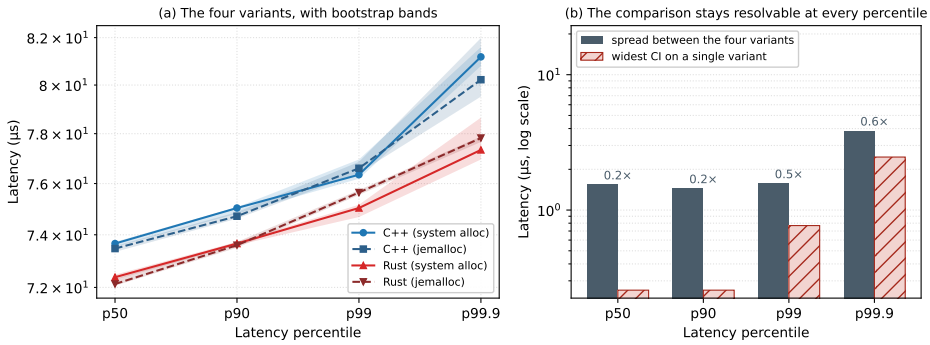


Figure 4.5: Per-operation latency across the percentile range at 56 kops/s. **(a)** The four variants as medians over 20 repeats, each with a bootstrap 95% band. The bands are the point: drawn as bare lines the curves would read as a claim of equivalence, where what the bands show is two languages separated at every percentile with the gap widening. **(b)** How far that separation can be trusted. The spread between the four variants is set against the widest confidence interval on any one of them, the ratio annotated above each pair of bars. Once it passes one, the uncertainty about a single variant exceeds the gap between all four and no ordering can be read off. It does not pass one anywhere here, peaking at 0.6 at p99.9, and on the virtualised host it passed one above the median. Both panels stop at p99.9, because both are built from per-run percentiles and the reported p99.99 is pooled instead, so its ordering in Table 4.1 is licensed by the millions of operations behind it rather than by this diagnostic.

4.4.2 Past saturation (RQ1)

RQ1 asks how the two languages compare under increasing load *and overload*, and everything so far has been below the knee. The operating point here is 784 kops/s offered, defined as the deepest swept load at which every variant has stopped meeting the offered rate, and the metrics are the two that remain interpretable there. Table 4.5 gives what each variant still sustains and Table 4.6 tests the comparison.

C++ sustains 147 kops/s where Rust sustains 84, a factor of 1.7. The paired difference is at least 62 kops/s under either allocator, it holds in 20/20 repeats, and its Holm-adjusted p sits at the corrected floor of the whole family of 14 tests. This is the largest language effect in the tested family and the most reproducible effect in the study. One larger ratio appears in the sweep, at the rung between the two languages' saturation limits (Section 4.4.1), and it runs the other way and is not tested.

The CPU column rules out the natural reading, that a variant completing less work is simply doing less, whether throttled, blocked or idle. Rust draws *more* total CPU than C++ here, reproducibly and under both allocators, having flipped the sign it carried at the representative load in Table 4.2. Divided through by the work that completed, its CPU cost per sustained operation is 1.8 times the other's, and neither variant is near the capacity of the machine, since both sit near six of the 16 pinned cores. The gap is therefore neither a capacity limit nor idleness. It is effort spent without completing

operations, which is the signature of a synchronisation path that spins, or parks and wakes repeatedly, where the other blocks once and sleeps. Figure 4.6 puts both halves of that reading side by side, and its grouping adds a third: the separation falls almost entirely inside the allocator groups, so the effect follows the variable the study manipulates and not the one it controls for.

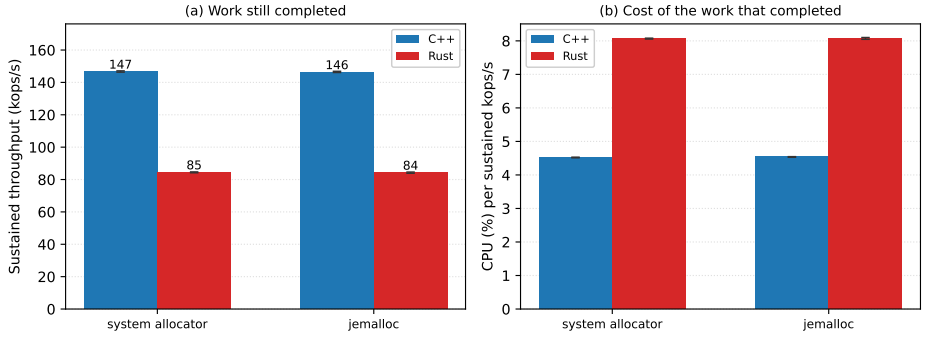


Figure 4.6: Behaviour at 784 kops/s offered, well beyond the 392 kops/s knee. Bars are medians over 20 repeats with bootstrap 95% intervals, grouped so that the two bars inside a group differ only in language. **(a)** What each variant still completes: C++ sustains a factor of 1.7 more than Rust. **(b)** Total CPU divided by the work that actually completed, which is what rules out the reading that the variant delivering less is simply doing less. It spends 1.8 times as much CPU per sustained operation, and the CPU figures are summed across threads.

Where that effort goes is narrowed by measurement. Past saturation the producers spend almost all their time in two places, the bounded blocking queue and the completion handle, and the microbenchmark below isolates those primitives and does not reproduce the full-service ordering at significance. That rules them out *alone* as the whole explanation for a 1.7× gap. As in Section 4.4.1, nothing in the safety model is implicated. The setting bounds the rest, and Section 5.5 says how far it travels: the depth of the collapse depends on one design, one request outstanding per producer and a queue bound of 1 024, measured with a harness whose producers block per request. What travels is the ordering under collapse, and that the isolated primitives do not produce it on their own. The closest prior work reports a comparable degradation past a threshold request rate and attributes it to the asynchronous runtime [33]. Section 5.2 takes up what it means that the same shape appeared here with no such runtime on either side.

Table 4.5: What each variant still sustains at 784 kops/s offered, well beyond the 392 kops/s knee: the median over 20 repeats above its bootstrap 95% confidence interval. Below saturation achieved throughput is pinned to the offered rate by construction and can express nothing about the languages. Here it is the only thing that can. CPU utilisation exceeds 100% because it is summed across threads, so 600% is six busy cores of the 16 pinned.

Variant	sustained throughput (kops/s)	CPU (%)
C++ (system alloc)	146.7	662.8
	[146.3, 147.2]	[662.5, 663.6]
C++ (jemalloc)	146.4	664.3
	[146.1, 146.9]	[663.3, 664.9]
Rust (system alloc)	84.5	681.8
	[84.2, 84.7]	[680.4, 682.5]
Rust (jemalloc)	84.3	680.3
	[83.9, 84.6]	[679.8, 681.5]

Table 4.7: Throughput through the bounded queue and completion primitives alone, with 7 contending producers and queue capacity 1 024. Values are medians over 20 interleaved repeats above bootstrap 95% confidence intervals. The paired test compares C++ against Rust on sustained throughput (kops/s). Like Table 4.4 it predates the move to bare metal and carries no environment record, so the comparison between the two alternately measured arms is what this section uses.

Language	Throughput (kops/s)	Mean ns/op	Paired p
C++	55.0	18178	—
	[53.7, 58.4]	[17110, 18632]	
Rust	58.7	17045	0.226
	[56.8, 61.8]	[16169, 17594]	

Table 4.6: Exact paired permutation test of C++ against Rust at 784 kops/s, the allocator held fixed, Holm-corrected across the same family of 14 tests as Table 4.2 rather than as a family of its own. "Repeats" counts how many of the 20 paired differences agree with the sign of the median, which a statistic computed from their mean cannot show. No latency percentile is tested here, for the reason fixed in advance in Section 3.5.

Metric	Allocator	Effect (Rust − C++)	Repeats	p	Holm
Sustained throughput (kops/s)	system	−62.2 [−62.9, −61.6]	20/20	0.0000019	0.000027
Sustained throughput (kops/s)	jemalloc	−62.2 [−62.7, −61.6]	20/20	0.0000019	0.000027
CPU utilisation (%)	system	+19.5 [+17.3, +19.9]	20/20	0.0000019	0.000027
CPU utilisation (%)	jemalloc	+16.8 [+15.6, +17.5]	20/20	0.0000019	0.000027

The contended queue and completion primitives on their own. The overload result attributes its gap to the two forced-idiom synchronisation constructs from the shape of the full-service measurement, so a microbenchmark isolates those constructs alone. 7 producer threads hammer one bounded queue of capacity 1 024 while a single consumer drains it, each submitting a no-op job carrying a one-shot completion handle and blocking until the consumer signals, with no `ShardCore` work in the path. C++ uses the hand-written `BoundedQueue` of Section 3.7 with `std::promise`/`std::future`, Rust `sync_channel` and a fresh `mpsc::channel()` per request. Each timed window lasts two seconds after a warmup, and 20 interleaved repeats are summarised as medians with bootstrap intervals and tested with the exact paired permutation procedure of Section 3.5. Table 4.7 reports the result.

The isolated primitives do not reproduce the overload ordering. C++ sustains 55 kops/s and Rust 59 kops/s, a gap of only 1.1× with $p = 0.226$, where the full service at 784 kops/s separates by 1.7× with its corrected p at the family floor. The overload divergence therefore involves extra work the microbenchmark strips away: the larger tagged `Request` on the C++ side, the worker's variant dispatch, and the map operations that still execute in overload. An orders-of-magnitude difference in the primitives alone is ruled out, which moves the search for the mechanism to the rest of the handoff path and not to the safety model.

4.4.3 Resource use (RQ3)

Table 4.8 reports resource use at the same sustained load. Peak resident set size separates the two *languages* cleanly, and whichever way the family is read: its paired intervals exclude zero, its effect keeps its sign under both allocators, and both rows clear the correction. It does not separate all four variants, because the C++ build with `jemalloc` and the Rust build with the system allocator land on the same footprint from opposite directions. Figure 4.7 plots the same measurement, and its middle panel is where this section's finding sits.

Two effects are present in it, not one, and they are of comparable size. Linking `jemalloc` costs 1.5–2.4 MiB in either language, and the Rust build costs 1.4–2.5 MiB more than the C++ build under either allocator, with a bootstrap interval of 1.2 to 2.6 MiB. Neither dominates. Part of the allocator figure comes from the fairness controls rather than from `jemalloc` in general: both builds pin the arena count identically, so that neither language's default could become an accidental advantage, and pinning it high makes the arena metadata a fixed cost on both sides. A deployment tuning `jemalloc` for footprint instead of for comparability would not pay it.

The relative size of the two effects is the one result in this chapter that a change of host overturned. On the virtualised machine the allocator effect measured 32.5–34.3 MiB, about 13 times the language effect rather than comparable to it, while the language effect was the same size there as here. So the *language* effect on peak RSS is reproducible across three machines and the *allocator* effect is not a constant of the allocators.

Every row in the upper block of Table 4.2 now clears the correction, including peak RSS under `jemalloc`, which did not on the previous host. That row failed there for a reason in the statistic rather than in the software: a handful of runs reported a resident set some 31 MiB low, and a permutation statistic built on a mean of paired differences is dominated by them. The same measurement here spreads by 1 MiB, so the contamination was the machine's. The rule fixed in advance in Section 3.5 for reading such a row against its interval was not needed, and it stays stated because a design that declares its reading rules only when they rescue a result is not applying them.

CPU utilisation behaves differently again, by regime rather than by noise. At this below-saturation operating point Rust draws slightly *less* CPU, by at most 5.1 percentage points, consistently under both allocators and clearing the correction under both, where Section 4.4.2 finds the sign flipped past saturation and the magnitude much larger. Achieved throughput adds nothing here, for the reason Section 3.5 gives: below saturation it is pinned to the offered load, and it is reported only to confirm that the point is sustained and not merely nominal, which it is to within 0.0%.

The memory finding is a difference in fixed cost and not in cost per context. At the 5 000 active UEs used in the sweep, the 8-byte layout difference of between the two context structures accounts for roughly 40 kB, two orders of magnitude short of the gap, so most of it is runtime and allocator-arena overhead, which does not scale with the workload and would matter proportionally less in a larger deployment. That is an argument from elimination and this campaign cannot do better, because it reads the resident set only through `getrusage`. A second bare-metal campaign reads the allocator's own accounting

on a host that also exposes hardware counters and decomposes the same gap directly (Table E.4). The RQ3 finding is therefore neither the flat "no resource penalty" that overlapping intervals would suggest nor a general one. Rust costs a few megabytes more of fixed footprint under both allocators, comparable to what the allocator choice costs here though not on every host measured, and its CPU cost changes sign with the load regime.

Table 4.8: Resource use at a sustained offered load of 56 kops/s: achieved throughput, peak RSS, and total CPU utilisation. The median over 20 repeats above its bootstrap 95% confidence interval. CPU separates the two languages outright. Peak RSS separates them at a fixed allocator, which is the comparison the paired test makes, and not across all four variants, because the allocator effect and the language effect are of comparable size and one middle pair therefore coincides.

Variant	throughput (kops/s)	peak RSS (MiB)	CPU (%)
C++ (system alloc)	55.99	11.1	124.1
	[55.99, 55.99]	[10.9, 11.2]	[123.4, 124.4]
C++ (jemalloc)	55.99	12.6	123.4
	[55.99, 55.99]	[12.5, 12.6]	[122.8, 123.7]
Rust (system alloc)	55.99	12.5	118.5
	[55.99, 55.99]	[12.4, 12.7]	[117.8, 118.9]
Rust (jemalloc)	55.99	15.0	118.3
	[55.98, 55.99]	[15.0, 15.1]	[117.6, 119.5]

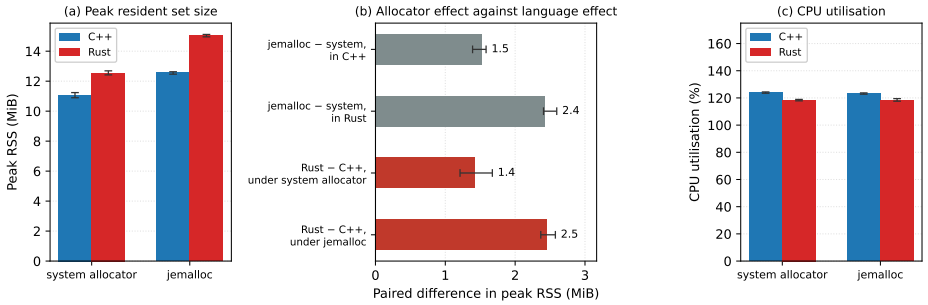


Figure 4.7: Resource use at a sustained offered load of 56 kops/s, grouped by allocator so that the two bars in a group differ only in language and the two groups differ only in allocator. Medians over 20 repeats with bootstrap 95% intervals. **(a)** Peak resident set size. **(b)** The two effects it contains, each as a paired difference over the same repeats: linking `jemalloc` costs 1.5–2.4 MiB, while the Rust build costs 1.4–2.5 MiB more than the C++ one. Both are real and on a shared axis they are of comparable size, which was not true on the host this study used first. **(c)** CPU utilisation, summed across threads, where the difference runs the other way and is small.

Which of these performance and resource findings belong to the code and which belonged to the machine is a separate question, and one a single-host study cannot ask. Two further campaigns bear on it, the virtualised host this study measured on first and a second bare-metal host at another design, and what survived each change is this: the overload divergence and the peak-RSS language effect did, the ratio between the language and allocator effects did not, and hardware counters on the second host turn Section 4.4.1's structural argument into a measurement.

4.4.4 Safety via fault injection (RQ2)

This study does not claim that Rust has no bugs. It injects the catalogue of 14 defects set out in Section 3.1, each as a small self-contained program flavoured with the subsystem's domain so that it maps onto a plausible bug in the service. The split into two groups is what the method of Section 3.3 turns on: seven classes from inside the region Rust claims to govern, and seven chosen precisely because safe Rust is expected *not* to prevent them. With the second group the study measures a boundary instead of demonstrating a claim, and a case that unexpectedly compiled, or unexpectedly failed to, surfaces against its recorded expectation instead of being absorbed into the conclusion.

Procedure. Each C++ defect is built and run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer, and the matching construct is written in safe Rust and compiled. Signed overflow is invisible to the first two sanitisers, so without its own column the integer-overflow class would be recorded as undetected in C++ when in fact it is caught. No static analyser is run, so the C++ columns of Table 4.9 report the tooling this study ran and not a ceiling. Where the Rust program compiles, it is run under a bounded timeout, because compiling alone would not establish whether the defect survives.

Results. The outcomes are shown in Table 4.9, measured with `g++ 13.3` (`-std=c++23`, `-O0` so the optimiser cannot elide an injected defect) and `rustc 1.97` (edition 2021). That C++ toolchain is the local development machine's and not the measurement host's `g++ 15.2.1`, deliberately, because whether a program compiles, and what a sanitiser reports when it runs, are properties of the toolchain and not of the machine hosting it.

Every one of the seven memory-safety and data-race defects is a compile error in safe Rust, where in C++ each is valid code whose defect shows up only at runtime, and only if a test exercises the buggy path. Each sanitiser covers a different part of the catalogue and no single one covers it, so the three between them still leave 4 of the 14 classes untouched, with a fifth reported only intermittently. The parenthesised codes are the `rustc` diagnostics that fire: conflicting mutable borrows (`E0499`), a mutable borrow while a shared borrow is live (`E0502`), use of a moved value (`E0382`), and a closure outliving borrowed data as it escapes to another thread (`E0373`). Two rows carry more than the other five. The use-after-move is the only in-guarantee class no C++ sanitiser reports at all: all three tools run clean while the acknowledgement goes out under an empty identity. A static analyser configured for it would plausibly flag the pattern, and `clang-tidy` ships such a check, whose documentation records that it follows control flow but not paths and considers only local variables and function parameters [40]. None is run here, for the reason given in Section 3.3. The thread-unsafe handle's diagnostic is `E0277`, a trait bound rather than a borrow error, because the rejection comes from `Send` and not from the borrow checker.

All seven boundary cases behave as predicted, and they mark out the edge of the claim. Every one compiles in safe Rust and every one misbehaves. The lost-update case ends in both languages with a silently wrong answer: against an expected total of 40 000 increments the safe-Rust program recorded 25 734 and the C++ program under Thread-

Sanitizer 27 767, losing 14 266 and 12 233. Neither language is meaningfully better off, which is the point of carrying the class at all. The deadlock hangs in both, the reference cycle leaks in both, the release-build overflow produces a wrong tunnel endpoint identifier (TEID) in both, and a failure inside one shard worker takes down more than that shard in both. The forgotten guard separates two things that resource acquisition is initialization (RAII) runs together whenever it is credited with preventing this. Rust removes the ordinary form of the bug, since the C++ version fails on a validation path that returns before a hand-written unlock and the Rust version has no unlock to skip, but it does not remove the class: both programs had to be killed by the timeout and neither language's tooling said anything.

The seam double-free is the double-free of the in-guarantee group, written the way a foreign-function boundary has to write it. Safe Rust rejects the double release with `E0382`, and handing a C caller a raw pointer is precisely the operation that ends the ownership tracking behind it. `Box::from_raw` then asserts ownership of whatever address it is given, so asserting it twice frees the storage twice. The C++ half of the pair is deliberately unremarkable and unchanged, so the only column that moves is the Rust one, and that boundary is the one Section 4.4.5 measures.

Two of the seven are cases where the C++ tooling is *ahead*. LeakSanitizer reports the C++ reference cycle as an unreachable allocation at exit, while the safe-Rust cycle leaks in silence, because leaking is safe in Rust by explicit design [41, 42]. UBSan reports the C++ overflow, because signed overflow is undefined there, while the Rust overflow wraps by definition at `opt-level = 2`. Defined-but-wrong is easier to debug than undefined-and-wrong and no less incorrect [43]. ThreadSanitizer does flag the inversion in C++, but on some runs and not others, 5 of 12 repeats in the run behind this table while the program deadlocks on every one, and the rate itself is not stable across runs. TSan builds its lock-order graph from acquisitions that *succeed*, so the cycle closes only when some thread completes an inversion without blocking. A positive control that inverts the same two locks without ever blocking is reported every time, which establishes that the detector was live.

The scope all of this exposes is precise. Rust's concurrency guarantee is the absence of *data races*. It is not a guarantee about lock-acquisition order, nor about the atomicity of a composed operation, nor about leaks, wrapping arithmetic, or fault containment, and it is suspended wherever a program writes `unsafe`. The lost update stays the quietest case, because no sanitiser reports it either.

Where the boundary classes are caught, and at what cost. RQ2 asks which defects are prevented, and also at what stage the rest are detected. For the boundary group the sanitiser columns leave that second half largely blank. Two Rust model checkers answer it, in the division of labour a production Rust system has been validated with: Loom explores interleavings soundly under bounded partial-order reduction, and Shuttle samples them probabilistically to reach cases beyond Loom [44]. Both report both of the original boundary defects, the deadlock as a closed cycle and the lost update as a failed assertion. Each is paired with a control in which the defect is fixed, by consistent lock order and by holding the read-modify-write in one critical section, and both controls come

out clean, so the reports come from the defects and not from the tools.

The cost is the part worth recording. Neither defect program runs under either checker as written. Each had to be re-imported against the checker's own synchronisation and thread types, because a tool that does not intercept the primitive cannot schedule it, and each had to be shrunk to model size: the lost-update model drops from twenty thousand iterations per thread to one, and the deadlock model loses the sleep that widens its window on real hardware. The answer for the Rust half of the boundary is therefore *detected at test time, after a rewrite*, while on the C++ half the sanitiser takes the program unchanged and finds one of the two unreliably and the other not at all. Neither language's story here is the simple one, and Chapter 5 develops the consequence for practice.

The seam double-free is answered by a different tool and comes with a sharper limit. Miri interprets the program against the Stacked Borrows aliasing model and reports the second release precisely, as a dangling box reconstructed from a freed allocation, while a control that releases the same handle once is clean, so the stage for this class is test time again and this time without a rewrite. The limit is what a migration has to weigh. Miri interprets Rust and cannot execute a C++ host, so the program it checked uses a Rust stand-in for the host. The tool that finds this defect cannot be pointed at the configuration the defect ships in, which is a seam with C++ on the other side of it. Nothing in the toolchain checks that boundary, and the practical answer is the one Section 4.4.5 argues from the prototype: keep the seam small enough to review by hand, because review is what is left.

A defect that was not injected. Every outcome above comes from a defect written on purpose, and that is the standing weakness of the method. Whoever assembles a catalogue already knows what safe Rust prevents. One case in this study does not carry that weakness. A design review of the specification found that its concurrency section forbade submitting an operation outside the service's running window (before `start()`, or after `shutdown()`), while neither implementation checked for it. The prohibition was addressed to the caller with no mechanism behind it, and in that gap the two languages did not fail alike. The C++ submit path indexes the shard vector that `shutdown()` has cleared, which AddressSanitizer reports as a heap-use-after-free. Called before `start()` it is a segmentation fault on the zero page. In Rust both calls were already a panic, from the ordinary bounds check on the same indexing operation, and nobody had designed for it.

What makes the case evidentially different is where it came from, not how severe it is. It sat in code written to be correct, in the arm whose safety claim rests on ThreadSanitizer, reachable through the public API by calling documented operations in a documented order, and it survived every check this study runs: 6 C++ test binaries, 27 Rust tests, 223 hand-written and 793 000 generated conformance operations, and three sanitiser configurations reported clean. It survived them because none of them ever called an operation outside the running window. ASan finds this defect only once a test drives it, which is the property the cost argument above assigns to runtime detection, seen here on the study's own artefact instead of a planted one.

The direction in which it must *not* be over-read is the flattering one. Rust did not prevent

Table 4.9: Defect-class taxonomy: measured outcomes for the injected catalogue. C++ under `g++ 13.3` with ASan/TSan/UBSan. Rust under `rustc 1.97`, edition 2021. "Compile error" means the safe-Rust program does not build, so the defect cannot be written. The lower block is the boundary group: defect classes chosen because safe Rust is *not* expected to prevent them, and does not. Two of those seven are the ones on which the C++ tooling is ahead. The body of this table is generated from the fault-injection taxonomy log (Section 3.6), so it cannot disagree with the run that produced it.

Defect class	ASan	TSan	UBSan	Safe-Rust outcome
Within the guarantee				
Use-after-free	detected	detected	not detected	compile error (E0499)
Double-free	detected	not detected	not detected	compile error (E0382)
Iterator invalidation	detected	detected	not detected	compile error (E0502)
Data race	not detected	detected	not detected	compile error (E0373†)
Unguarded shared mutation	not detected	detected	not detected	compile error (E0502†)
Use-after-move	not detected	not detected	not detected	compile error (E0382)
Thread-unsafe handle shared	not detected	detected	not detected	compile error (E0277)
Boundary of the guarantee				
Lock-order deadlock‡	not detected	intermittent§	not detected	compiles, deadlocks
Lost update	not detected	not detected	not detected	compiles, wrong result
Reference-cycle leak	detected	not detected	not detected	compiles, leaks
Integer overflow (release)	not detected	not detected	detected	compiles, wrong result
Worker-failure blast radius	not detected	not detected	not detected	compiles, shard outage
Double-free across the seam	detected	not detected	not detected	compiles, aborts
Forgotten lock guard‡	not detected	not detected	not detected	compiles, deadlocks

† Several diagnostics fire for these cases. The first is shown (the data race also raises `E0382` and `E0502`, the unguarded mutation `E0524` and `E0597`). ‡ On these rows the instrumented binary was killed by the harness timeout rather than reporting anything. Nothing is corrupted and nothing is undefined, so the tools that detect corruption have nothing to say. The forgotten guard is silent under all three, and the lock-order case is the one TSan sometimes catches, as the next note records. § ThreadSanitizer reports the lock-order inversion on some runs and not others, measured at 5 of 12 repeats, while the program deadlocks on every one. The detector builds its lock-order graph from acquisitions that *succeed*, so a run whose first inversion blocks never completes the second edge of the cycle.

this at compile time, and nothing in the borrow checker knows that a service must be started before it is used. What turned undefined behaviour into a diagnosable failure was a slice bounds check, a runtime guarantee and so the same kind of assurance the C++ sanitiser offers, not the compile-time kind Table 4.9 records. The honest claim is that checking by default turned a specification gap into a panic, not that a type system caught a lifecycle error. What remains is a panic naming its cause on every build against a heap-use-after-free that needs an instrumented binary and a test that thinks to look. Both implementations now reject the call explicitly, in both directions, and both suites cover it.

Figure 4.8 draws the same 14 outcomes as a matrix, where the change of character at the boundary rule is the shape of the picture and not something to be read off row by row. Both it and Table 4.9 are generated from the fault-injection taxonomy log of Section 3.6, so neither can drift from the other.

4.4.5 Migration (RQ4)

Incremental adoption is assessed with a working prototype, built in the direction a real migration takes: an existing C++ host keeps a Rust component behind a C ABI. The Rust service is exposed through a thin `extern "C"` seam in a *separate* crate that depends on

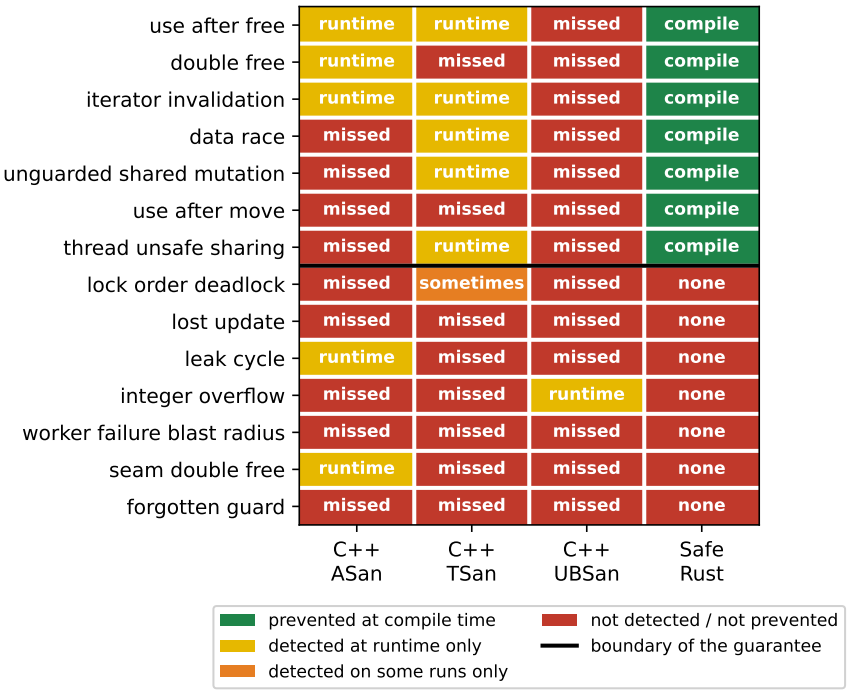


Figure 4.8: The outcomes of Table 4.9 drawn as a matrix, generated from the same taxonomy log (Section 3.6). Above the rule the Rust column is a single colour: every defect is a compile error, so none can be written. The C++ columns there are a patchwork instead, and no one sanitiser covers them. The use-after-move row is the one no sanitiser covers at all. Below the rule the Rust column joins them, and on the reference-cycle leak and the release-build overflow the C++ tooling is ahead. The lock-order deadlock is the one cell where a tool reports intermittently: ThreadSanitizer flags the inversion in 5 of 12 runs while the program deadlocks on every one.

the unmodified core library, so adopting the component needs no change to the component itself. The C++ host sees only an opaque handle and a set of plain-C entry points (a hand-written header, no generated bindings), and it drives the full UE lifecycle across the boundary. The error model crosses intact, so a duplicate connection setup returns `InvalidState` through the seam. Figure 4.9 shows the resulting structure and where the `unsafe` sits.

Table 4.10 summarises the interop cost. The seam is small, 11 C-ABI entry points in 231 lines of Rust plus a 66-line C header with no third-party FFI dependency, and its safety boundary is clean: the core library stays free of `unsafe` and the raw-pointer handling the C ABI requires is confined to the seam crate, in its 9 `unsafe` functions, under a single documented safety contract. The C++ side never sees a Rust type.

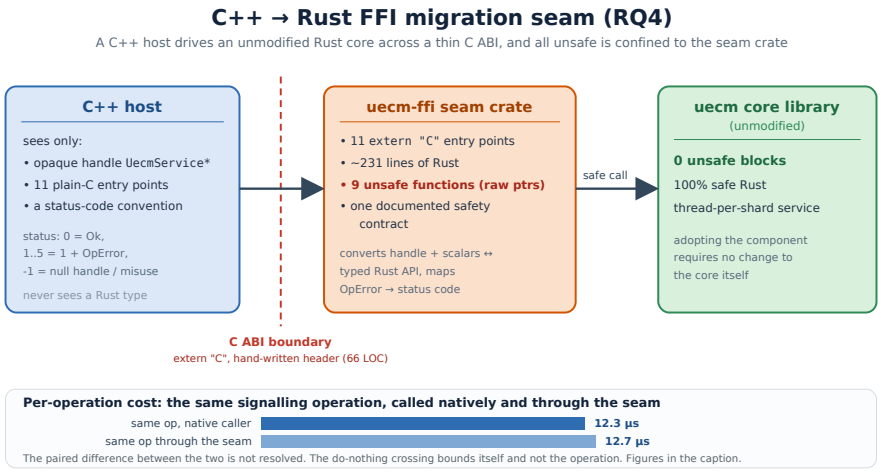


Figure 4.9: The C++-to-Rust FFI migration seam. The C++ host sees only an opaque handle, 11 plain-C entry points, and a status-code convention. A thin `extern "C"` seam crate concentrates all of the raw-pointer `unsafe` (9 functions) and calls the unmodified Rust core, which remains free of `unsafe`. The band along the bottom is the comparison the argument rests on: the same signalling operation called natively at 12.3 µs and through the seam at 12.7 µs, whose paired difference is not resolved and is bounded at 10.91% of one operation. The two bars are meant to look alike. The do-nothing crossing costs 2.54 ns and bounds only itself.

Table 4.10: FFI migration prototype: seam size, safety boundary, and per-call interop cost (C++ host calling the Rust component across a C ABI, `-O2`). The core library is unmodified and `unsafe`-free. All interop is confined to the seam crate. The last three rows are the direct comparison: one signalling operation on the same service configuration and the same loop, invoked natively and then through the seam, so the difference isolates what the boundary adds rather than what the operation costs. Like the other paired experiments it predates the move to bare metal and carries no environment record, so the bound is a ratio between two alternately measured arms and not a claim about a host.

Metric	Value
C-ABI entry points	11
Seam size (Rust `lib.rs`)	231 LOC
Hand-written C header	66 LOC
`unsafe` functions in core library	0
`unsafe` functions in FFI seam	9
Raw FFI crossing	2.54 ns/call
Signaling op, native caller	12.3 µs/call
Signaling op, through the seam	12.7 µs/call
Seam cost per operation (paired)	+192 ns [−954, +1341]
as a fraction of the operation	+10.91% at the upper bound

The per-call cost is measured twice, and only the second measurement answers RQ4's question. The first is the crossing itself: a do-nothing `extern "C"` call costs 2.54 ns, an ordinary indirect call, negligible next to the microsecond-scale round-trip of a signalling operation. That bounds the crossing and says nothing about what the seam does to a real operation, whose round-trip is dominated by a cross-thread handoff, and an argument that the handoff "would be paid identically anyway" is reasoning standing in for a measurement.

The measurement is therefore made directly. The same signalling operation, on the same

service configuration and in the same loop, is invoked natively from Rust and then from C++ through the seam, interleaved over 15 repeats so the pair is matched. The paired difference is +192 ns per operation, with a bootstrap interval of $[-954, +1341]$ ns and an exact paired p of 0.239. No cost is detected, and more usefully the interval bounds any cost the measurement could have missed at 10.91% of the operation. The interleaving is not optional here. A single pass of each, run while preparing this section, made the seam look *faster* than calling natively by a third, because the run-to-run spread of the handoff on a shared host is far wider than anything a C ABI could contribute. Figure 4.10 shows the repeats behind those numbers, and its left panel is that spread: the pairs cross, over a range an order of magnitude wider than the difference under test.

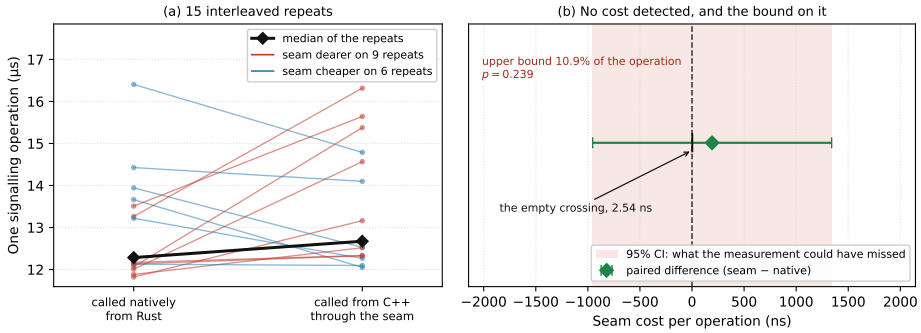


Figure 4.10: The interoperability cost measured on a real operation rather than inferred from an empty call. **(a)** The same signalling operation invoked natively from Rust and from C++ through the seam, one line per interleaved repeat, with the medians in black. **(b)** The pairs reduced to their difference: +192 ns per operation with a bootstrap 95% interval of $[-954, +1341]$ ns and an exact paired p of 0.239. No cost is detected, and the upper end of that interval is what bounds the cost the measurement could have missed at 10.91% of the operation. The empty `extern "C"` crossing, 2.54 ns, is marked for scale rather than as the answer: it is indistinguishable from zero on the axis the operation is measured on.

The main engineering effort therefore goes into *designing* a C-compatible API surface, with an opaque handle, scalar-typed operations and a status convention, and not into runtime overhead. The reverse direction (Rust calling into C++) is symmetric, but it needs a C-ABI wrapper on the C++ side and it brings `unsafe` back to the Rust caller. The prototype and its measurements inform the migration guidelines of Chapter 5.

RQ4 asks what adoption costs in effort, interoperability and risk, and the seam answers only the middle term. Three further axes a team asks about first were measured, and each complicates a folk version of the answer. Build time survives the objection only in the aggregate: the full Rust build costs 1.8 times the CPU `g++` needs, but 54% of that is third-party code and the project's own source compiles in 0.8 times what `g++` spends on its equivalent. The shipped footprint ordering reverses once the linkage policy is held fixed instead of left at each toolchain's default. And the compared component resolves 0 crates with 0 `unsafe` sites, so the transitive-dependency objection does not reach the artefact that would ship. Build time is the one axis whose absolutes belong to the measuring machine, which is why it is reported as a paired ratio between two toolchains timed alternately.

5 Discussion

5.1 Summary of findings

Table 5.1 collects the headline finding for each research question with the strength of the current evidence. The safety results (RQ2) do not depend on the measurement environment. The migration results (RQ4) depend on it along one axis only, which is why build time is reported as a paired ratio. The performance and resource results do, and they now resolve on the host of Section 4.1, including the far tail below saturation, which the same design on the earlier host could not resolve.

Table 5.1: Summary of findings per research question and the current strength of evidence.

RQ	Headline finding	Evidence status
RQ1	The answer divides at saturation. Below it the variants agree to 2 μs (2.1%) at the median, and the far tail resolves in Rust's favour, the pooled p99.99 of the two Rust variants sitting below both of the others by up to 87 μs. The end-to-end number is only 0.250% operation, so it cannot bound a language effect in any case, while the operation measured apart is 1.6× faster in Rust. Past saturation they diverge decisively and in both directions: the Rust arms hold their offered rate one rung further, to 448 kops/s against 392, while at 784 kops/s, where all four have collapsed, C++ sustains 1.7× the throughput of Rust in 20/20 repeats under both allocators.	Overload, operation and far tail: resolved, and each reproduced at a second shard count. Mechanism: tested. Primitives alone insufficient. The saturation-limit gap is descriptive, not tested
RQ2	All seven injected memory-safety and data-race classes are prevented at compile time in safe Rust, where in C++ each compiles and is caught, if at all, only at runtime and by no single sanitiser. The guarantee stops there: the seven boundary classes compile in both languages, and on the leak and the release-build overflow the C++ tooling reports what Rust's does not.	Final
RQ3	A memory penalty that reproduces: 1.4–2.5 MiB more peak RSS in the Rust build under *both* allocators, in the same direction and order of magnitude on all three machines measured. The allocator's own 1.5–2.4 MiB is comparable here and was an order of magnitude larger on the virtualised host, so their ratio does not travel. CPU inverts across the regimes: at the representative load Rust draws up to 5.1 percentage points less, and past saturation 1.8× more per completed operation.	Memory: resolved, and located outside the allocator by its own accounting. CPU: resolved in both regimes, opposite signs, and one sign below saturation on the second bare-metal host
RQ4	The seam is small and keeps the core `unsafe`-free at compile time. Measured against the same operation called natively, no cost is detected and any tax is bounded at 10.91% of an operation. The real cost is API design. The core remains exposed at run time to memory errors in the C++ host it shares an address space with. On the adoption axes beyond the seam, 54% of the Rust clean build is third-party code and the project's own code compiles in 0.8 times the CPU `g++` needs, the footprint ordering reverses with the linkage policy, and the compared component resolves 0 crates.	Final. Build-time absolutes are host-bound, the ratio is not

5.2 Discussion on the research questions

RQ1 (predictability). The two load-dependent effects point at opposite languages, so a single verdict would be false. A service sized to run below its knee should prefer the arm that holds its offered rate further, to 448 kops/s against 392. One that a signalling storm will drive past every knee should prefer the arm that still completes more work afterwards, by 1.7 at 784 kops/s, at the corrected floor of the whole test family, while the slower arm spends more CPU on each operation it does complete (Section 4.4.2). Only the second is tested, the first being descriptive for the reason Section 4.4.1 gives. A comparison stopping at nominal load, as most in this literature do, would have concluded the opposite with the same apparatus.

What a deployment sizes its margin from is the far tail. At the representative load the median gap is a fraction of a 70 μs operation, while the pooled p99.99 is an ordering in Rust's favour whose extremes differ by 87 μs, roughly a whole median operation, reproduced at a second shard count on a second machine. The median result is consistent with the wider evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [12, 45], and weaker than it looks: the operation the languages execute is 0.250% of what the sweep measures (Section 4.4.1), so an end-to-end comparison in a design dominated by a cross-thread wakeup describes the design and not the languages. Measured on its own, the operation differs by 1.6×.

The overload shape reframes the closest prior work instead of merely agreeing with it. Arvidsson and Kaller measure this quantity for 5G control-plane procedures and find Rust's 99th percentile degrading faster beyond a threshold request rate, traced to stackless `async/await`, which permits a context switch immediately before an I/O operation begins [33]. Section 2.3 accepts that attribution, and this study was built to exclude the confound behind it: one worker per shard over a bounded blocking queue, and no asynchronous runtime on either side. Section 1.3 committed in advance to what each outcome would mean, and the second branch fired. A degradation past a threshold rate appeared with the concurrency model held fixed, so the asynchronous runtime cannot be the only place such an effect starts. The prediction also named where to look first, bounds and overflow checking [46, 47], and that part was wrong. The effect appears only once the handoff path is contended, which points at the two standard libraries' blocking primitives, and Section 4.4.2 finds that those primitives alone do not reproduce the full-service ordering at significance. The cause is extra work on the C++ handoff path and not the safety model.

RQ2 (safety). The result does not depend on the environment, and what makes it interesting is the half that bounds the expectation and not the half that confirms it. Within its scope the difference from C++ is one of kind and not of degree: a compiler that refuses to build a program rules the defect out of every execution, where a sanitiser rules it out only of the executions an instrumented test happens to take. What is absolute is the execution coverage, not the breadth of the catalogue, which is still the 14 programs written for it. That fits the Common Vulnerabilities and Exposures (CVE) evidence that Rust memory-safety bugs always involve `unsafe` [48] and the caution that unsafety can

spread through dependencies [49], which is why keeping the core `unsafe`-free matters. The seam double-free is that evidence under controlled conditions, the one class safe Rust rejects and unsafe Rust does not.

The one case reaching past the written programs is the lifecycle defect of Section 4.4.4, found in the implementation rather than written for the catalogue, and it revises the picture in two directions. It strengthens the cost argument, because a real defect in code written to be correct survived tests, parity vectors and three sanitisers purely because nothing exercised the path, and runtime detection that depends on test coverage is weaker in practice than it looks in a table where every row is driven deliberately. It also narrows what may be credited to Rust, because the contribution was a bounds check and not a proof: the guarantee worth budgeting for is *checked by default* and not *ruled out at compile time*. The root cause belongs to the method rather than to either language. A specification stating a prohibition without assigning a mechanism leaves each implementation to fail in its own idiom, and the two idioms here are a panic and a use-after-free.

Where the guarantee stops matters just as much to a vendor deciding what Rust buys. The lock-order deadlock shows that Rust reasons about *aliasing* and not *acquisition order*: a `Mutex` guard is an ordinary value whose lifetime the borrow checker tracks perfectly, and tracking it perfectly says nothing about whether two threads take their locks in compatible orders. The tooling confirms this is a boundary and not an artefact of the injected example, since static deadlock detection for Rust is an active research problem [50] and model checkers for Rust kernel code are motivated explicitly by ownership and borrowing leaving data races, deadlocks and livelocks reachable [51]. The lost update is sharper. It escapes ThreadSanitizer for precisely the reason it escapes the borrow checker (Section 4.4.4): with every access correctly synchronised there is no unsynchronised access to find. Nor is the class marginal. In a field study of 105 real concurrency bugs, about a third of the non-deadlock cases violated the order the programmers intended rather than their locking, and the authors record that such intentions are not readily expressible through locks at all [52]. The distinction between a data race and a race condition is the one "thread safety" routinely runs together, and Rust removes only the first.

Adopting Rust therefore retires one category of concurrency defect completely and leaves another untouched, and the one left untouched fails silently, corrupting a counter or a state decision instead of crashing. A migration plan treating "memory-safe" as "concurrency-safe" would mis-budget its review and testing effort, which is why this boundary is reported as a first-class result and not as a caveat. Figure 5.1 draws it, with the mechanism that does the enforcing, or the absence of one, named in each panel.

RQ3 (overhead). The answer splits by resource, and the splits do not all point the same way. On memory there is a penalty: the Rust build carries a few MiB more peak resident set and keeps that sign under both allocators. Two things make it credible beyond the p-value. Peak RSS is a run-level maximum and not a tail statistic, so the scheduling jitter that destroys the p99 does not touch it, and it reproduces on a second machine with a different compiler, which no other timing-adjacent result here can claim. The useful way to size it is against the allocator's own effect, and that comparison is the part which does *not* travel. The two are of comparable size on the reported host and on the second

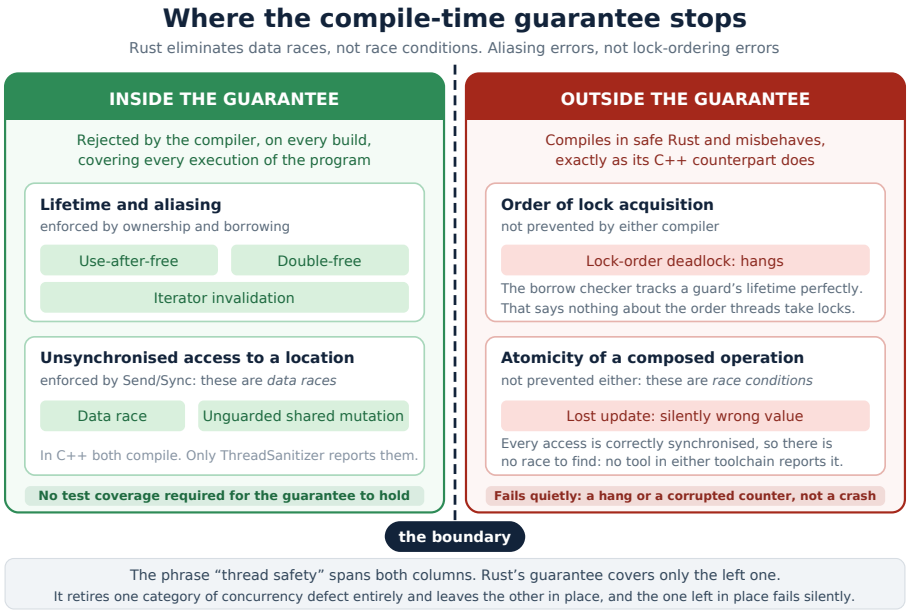


Figure 5.1: Where the compile-time guarantee stops. On the left are the classes safe Rust rejects outright, grouped by the *mechanism* that rejects them rather than by defect family: ownership and borrowing for lifetime and aliasing errors, which is why iterator invalidation sits with them, and the `Send`/`Sync` discipline for unsynchronised access to a location. On the right are the classes the catalogue included to find the edge, which neither compiler prevents. The diagram names one class per mechanism and is not the catalogue itself: the use-after-move belongs with the ownership cases on the left and the shared thread-unsafe handle with the `Send`/`Sync` ones, while the leak, the release-build overflow, the worker-failure case, the seam double-free and the forgotten guard sit alongside the two on the right. The phrase "thread safety" spans both sides of the line, and the guarantee covers only the left. The line itself moves once more than the diagram can show, because a program that writes `unsafe` crosses it voluntarily.

bare-metal one, while on the virtualised host the allocator's was an order of magnitude larger, because the allocator effect shrank by more than ten times across the change of machine and the language effect did not move. The language effect replicates and its ratio to the allocator's does not, so whether a few MiB of language choice is more or less than the allocator choice is a question about the machine.

On CPU the answer depends on the load, and that dependence is the finding. Below saturation Rust draws slightly *less* under both allocators, and past saturation the sign flips and the magnitude grows (Section 4.4.2), so a single number for "the CPU cost of Rust's safety model" would average two regimes behaving in opposite directions, and neither figure is the safety model in any case. Throughput splits the same way: below saturation there is nothing to measure, because achieved throughput tracks offered load by construction, and above it sits the largest effect in the study.

Three things bound what the memory penalty licenses. It is a fixed cost rather than a per-context one, so its share of a deployment's footprint falls as the deployment grows. It is located *outside* the allocator by the allocator's own accounting, since the Rust build holds less live data and fewer resident bytes inside `jemalloc` and still peaks +1.74 MiB higher, which makes it a one-off charge per process for the standard library and the runtime rather than a tax on the work. And nothing here isolates bounds checking or ownership-driven copying as the cause: the mechanisms the literature has localised [46, 47] are candidates to test, not conclusions to import. Adopting Rust here costs a few

megabytes of fixed footprint per process and buys a faster core operation (Section 4.4.1) that this workload's structure never turns into throughput below saturation, and that turns into a deficit above it.

The sign of that result is worth defending, because one of the closest prior works reports the opposite sign while two others agree with it. A second Ericsson-hosted study of RAN node management software found Rust's peak memory usage generally higher than C's [31], and a matched pair of user-space device drivers outside telecom put the Rust version's footprint about 29% above the C version's [53]. Neither separates the allocator from the language, the distinction Section 4.4.3 rests on, so they support the direction and not the split.

The disagreement is more instructive. Karlbäck and Orö measure peak resident set on a RAN workload and find Rust the *lowest* of three languages and C++ the highest, with the gap widening as concurrency rises, a pattern they call surprising and attribute tentatively to the initial stack size of the green threads [30]. The two findings are compatible once it is clear they measure different quantities. Their C++ runs on Boost.Fibers, which allocates a stack per fiber, so ten thousand concurrent sequences cost ten thousand stacks, while their Rust runs on Tokio, where a task is a heap-allocated future of a few hundred bytes. Their figures capture the per-context cost of the concurrency runtime, which is why it grows with the number of contexts. The design measured here has no green threads on either side, and a fixed thread-per-shard pool leaves no per-context allocation to differ, so what remains is a fixed footprint constant in the number of UEs. Their disagreement is between two concurrency runtimes and not between two languages, which is as direct a demonstration as the literature offers of why this study holds the concurrency model fixed. It also sharpens the caveat on the present number, which licenses nothing about Rust's memory behaviour under a task-per-UE design.

RQ4 (migration). Incremental adoption is technically cheap. A C++ host can call an unmodified Rust core across a thin C-ABI seam whose cost on a real operation is bounded at 10.91%, and all `unsafe` stays inside the seam. That bound comes from the matched comparison of Section 4.4.5 and not from setting an empty crossing against a full round-trip, which are different experiments: on this host a single unpaired pass of each was enough to flip the apparent ordering. The dominant cost is human, not machine, designing a C-compatible surface and, as the adoption studies report [54, 55], climbing Rust's learning curve. That makes migration risk organisational and not a performance concern, and the two largest documented adoptions into existing C and C++ codebases report the same ordering of costs [56, 57].

One boundary has to be drawn explicitly, because the natural reading overstates it. "All `unsafe` confined to the seam" is a claim about what the compiler verifies, true as measured, with no `unsafe` in the core crate and every raw-pointer operation in 9 reviewable functions. It is not a claim that the core is protected at run time. Host and component share an address space, and a memory error on the C++ side can corrupt Rust-owned data however carefully the Rust side was checked, which is why the remedies proposed in the literature isolate the two heaps with hardware support instead of relying on interface discipline [58, 59], and why the defect classes arising at such boundaries have been catalogued in their

own right [60]. A migration of this shape buys a component that stops *contributing* new memory-safety defects while it stays exposed to those of the host, which is a reason to sequence a migration so that the components sharing the most mutable state cross over together.

The three axes measured beyond the seam complicate two folk versions of the same conclusion. Build time survives only in the aggregate: the full build does cost 1.8 times the CPU, but 54% of that is third-party code, so the project's own source compiles for 0.8 times what `g++` spends on its equivalent, and an organisation worried about build time should look at its dependency budget before its compiler. The edit-compile loop is where C++ genuinely leads, and it leads unevenly: its advantage holds for edits inside a translation unit and reverses for edits to a widely included header. Binary size behaves worse than an argument would predict. The 6.8× figure that gets quoted compares two linkage policies, and the ordering depends on which policy is held. The axis that matters most for a regulated codebase has the cleanest answer: the component compared here resolves no third-party code and contains no `unsafe`, so the transitive-dependency argument does not reach the artefact under discussion.

5.3 Revisiting the hypotheses

Section 1.3 committed to three hypotheses before any measurement was taken. Stating their disposition explicitly is what distinguishes a test from a description.

H1: supported below saturation, refuted above it. The hypothesis had two parts: that the languages are performance-equivalent below saturation, and that any difference is confined to the far tail. The first is supported. The second is refuted, and not in the direction the wording anticipated, for the reasons Section 5.2 gives. The largest difference the study found lies in overload, which H1 did not consider at all, so the hypothesis as written would have led the campaign to look in the wrong place. The result was found because the sweep was pushed past saturation on methodological grounds rather than in expectation of it. The clause H1 did name is half right. For most of this study the far tail could not be resolved at all, so that part was untestable rather than refuted. It is resolved now, in Rust's favour, at loads where the median difference is about a microsecond on an operation of 70 μs, and a second bare-metal campaign agrees on the direction. H1 was right that the far tail is where a below-saturation difference would show up, and wrong that it is the only place any difference shows up. A second disposition is that the hypothesis is not well posed. "Performance-equivalent" was written as though it named one quantity, where Section 4.4.1 shows it names at least three: the delivered operation below saturation, the code the compilers emit, and sustained work under overload. Those three do not move together.

H2: supported in its precise form, refuted in its loose one. The hypothesis was that safe Rust prevents at compile time "the memory-safety and concurrency defect classes that C++ leaves to best-effort detection". For the seven injected classes of that kind it holds without exception. The fault-injection study still refutes the

broader reading the wording invites: the seven boundary classes are exactly the sort C++ leaves to best-effort detection, safe Rust does not prevent them either, and on the leak and the release-build overflow the C++ tooling is the one that reports. H2 is therefore accepted for memory safety and data-race freedom, and rejected for concurrency correctness at large. It also carries an unstated precondition the seam case makes explicit: the hypothesis is about *safe* Rust, and the same class it rules out returns as soon as a program writes `unsafe`. The second reading is the one practitioners tend to assume.

H3: supported, and tested the right way on the second attempt. The hypothesis was that interoperability cost is negligible against the operations invoked. Measured directly (Section 4.4.5), no cost is detected and any per-operation tax is bounded at 10.91%. What supports the hypothesis is the bound and not the failure to detect a difference. The first version of this measurement set an empty crossing against a full round-trip and inferred the answer, comparing two different experiments, which would have given the right conclusion for the wrong reason. The accompanying claim, that the real cost is engineering effort, is supported structurally and not statistically: the seam is small and mechanical, but designing its C-compatible surface was the part of the prototype that took actual design decisions.

5.4 Engineering guidelines for incremental adoption

Appendix G records seven guidelines in the order a first migration would run them, each tied to the finding that supports it and to the limit of that support. A guideline quoted without its limit is how the RQ2 result becomes "the migrated component is safe" and the RQ4 result becomes "migration is cheap", and this study establishes neither. Two belong in the running text because the study arrived at them late. An end-to-end request/response measurement of this design is 0.250% language and the rest scheduler (Section 4.4.1), so below saturation it returns "no difference" between almost any two competent implementations, and a vendor running its own comparison should budget for the end-to-end measurement, which says what a deployment will feel, and the component measurement, which says what the language is doing. Every conclusion drawn while the sweep stopped at nominal load was a null, correct at that operating point and silent about the one where the two implementations differ by 1.7× (Section 4.4.2). The reusable artifact is the benchmarking framework and not its current output.

5.5 Where this study sits, and what it cannot support

The studies closest to this one are the five vendor-hosted theses of Section 2.3, whose verdicts contradict each other in every direction. The one reporting the opposite sign on peak memory reconciles with this study once its arms are read carefully: its C++ arm allocates a stack per concurrent sequence where its Rust arm does not. The contrary results matter as much, the matched comparisons in which safe Rust is measurably slower, and they qualify the "zero-cost" framing without settling it.

Three threats to validity are load-bearing and belong in any reading of the results above, alongside the controls of Section 3.4 and what each leaves behind. One person wrote both implementations, so single-author skill asymmetry cannot be ruled out, and the same failure mode reversed a published verdict in this industry. The end-to-end instrument is the wrong one for part of what RQ1 and RQ3 ask, which is why Section 4.4.1 measures the operation apart from the handoff. And the overload divergence, the largest effect in the study, rests on an attribution the campaign narrowed but did not localise.

5.6 Ethical and professional reflection

The study processes no personal, subscriber, or operator-confidential data. All traffic is synthetic and the subsystem is modelled on production code rather than copied from it, so the artifacts can be released openly. The *use* of the study is a different matter, because a language-selection recommendation in this domain is not a neutral technical output. Two responsibilities follow. The first concerns over-claiming. A vendor could read the RQ2 result as licence to describe a migrated component as “safe”, and the boundary cases are why that reading would be wrong: a deadlocked control plane is an outage, a silently lost counter update can misinform a mobility decision, and nothing demonstrated here prevents either. Reporting the boundary as prominently as the guarantee is a professional obligation, because the audience for this work is making a decision that affects networks people depend on. The second concerns how far the evidence reaches. Tail-latency numbers from any single machine are a narrower basis for a production commitment than they look, which is why Table 5.1 carries an evidence status per research question and why the earlier virtualised campaign is reported rather than quietly discarded. A migration recommendation also carries organisational consequences, and the cost moved from run time to human effort rather than disappearing.

6 Conclusion and Future Work

This thesis compared modern C++ and Rust on a UE Context Management service modelled on a real Nokia gNB-CU-CP component, along four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and incremental-adoption cost. Built against a single frozen specification and driven through identical code paths, the two implementations isolate language effects from design and skill differences.

Three conclusions follow from the evidence. First, on safety the difference is one of *kind*, and it has a clear boundary. Safe Rust rejects all seven injected memory-safety and data-race classes at compile time, and one of those seven is a class no C++ sanitiser reports at all. Hazards C++ can only hope to catch under an instrumented test become unwriteable. The edge stops there. All seven boundary classes compile in safe Rust and all seven misbehave, though not always in the same way as in C++, and on two of them the C++ tooling is the one that reports. The guarantee is therefore best described precisely. Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors, leaks, wrapping arithmetic or fault containment, and it eliminates none of them in a region marked `unsafe`. Where the boundary defects are caught, they are caught at test time, in two of the three cases only after being rewritten into a model checker's primitives, which is a real cost and a different one from the sanitiser's.

Second, on migration the cost is dominated by API design and the learning curve, not by any run-time tax. An unmodified Rust core can be driven from C++ across a thin, `unsafe`-isolated seam whose cost on a real operation, measured against the same operation called natively, is bounded at 10.91%. The objections a build engineer raises first do not change that ordering. Most of the Rust build's extra expense is compiling dependencies and not the compiler's own work, the binary-size gap is a linkage policy and the ordering depends on which matched policy is chosen, Rust smaller when both standard libraries are linked statically and C++ smaller when both are resolved dynamically. The compared component resolves no third-party code at all. Where C++ leads is the edit-compile loop, and it leads only for edits that stay inside a translation unit.

Third, on performance the evidence first divides by what is being measured. An end-to-end operation in this design is 0.250% the work the language performs, and the rest is a cross-thread handoff the operating system performs, so an end-to-end null cannot bound a language effect at all. Measured on the operation alone, Rust is the faster of the two, by 1.6×, for reasons that trace to the standard library's hash-map layout and not to anything about safety. At the representative load the paired test resolves 1.3 μs in Rust's favour under both allocators, too small for a deployment to care about at the median, while at p99.99 the slowest Rust variant sits 50 μs below the fastest C++ one. The Rust arms then hold their offered rate to 448 kops/s against 392. Beyond that, the C++ build sustains 1.7× the throughput of the Rust build, in 20/20 interleaved repeats under both allocators, while the Rust build spends 1.8× the CPU on each operation it completes, so what it loses is not idleness but effort that finishes nothing. The isolated primitives do not reproduce that ordering at significance (Section 4.4.2). On memory the Rust build

carries a few MiB more fixed footprint under both allocators, in the same direction and the same order of magnitude on every machine measured, while the cost of the allocator choice moved by more than tenfold between them.

Taken together these support a narrow claim. For stateful, concurrent, partitionable control-plane services of this kind, at the loads where the service still meets its offered rate, the choice is which classes of defect an organisation wants to make unwriteable, and whether it will pay the engineering and learning cost of getting them. Past that point it is a performance decision as well. The reasons for the latter lie in the concurrency plumbing and not in either language's guarantees. What this thesis adds is an instrument and not a verdict: the frozen specification, the parity-tested pair, a measurement framework that reaches past the saturation knee, a decomposition that says how much of what it measures the language is answerable for, and a defect catalogue that maps where the guarantee stops.

Several directions extend the work. The immediate priority needs no new machine: vary the transparent-hugepage policy on one host with everything else held fixed. Three hosts agree that the Rust build carries the larger footprint and disagree about how that compares against the allocator cost. The hugepage policy differed between those hosts and only the system-allocator arm inherits it. A single machine and two settings settle whether a footprint claim of this kind belongs to an allocator or to a kernel setting. Beyond that, the framework can take captured anonymised traces in place of the synthetic workload, further control-plane subsystems or the data-heavy layer-2 and physical layer (PHY) path, and a whole-program model checker covering foreign dependencies [61]. It can also compare Rust async runtimes against the thread-per-shard model, which would separate the tail divergence Arvidsson and Kaller attribute to `async/await` [33], measure in-process isolation of the two heaps at the seam [58, 59], and track defect density and review effort as Rust is adopted in a production RAN codebase.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, through the Cursor development environment, is the only generative AI system used in preparing this thesis, from June 2026 until submission. It is named neither as an author, which University of Helsinki policy forbids, nor as a source, which is the author's own choice.

What the assistance covered. It was extensive. The model drafted and revised the running prose and proposed the order in which the arguments are made. It wrote and refactored both implementations of the UE Context Management service, the load generator, the fault-injection catalogue, the interoperability prototype, the analysis code behind the bootstrap intervals and the paired permutation tests, and the figure-generating, build and LaTeX sources. It also suggested candidate literature. The abstract is the one part it must not touch, because it is this programme's maturity test, and University of Helsinki policy never permits a language model in one. The abstract was accordingly written by the author unaided.

What the model did not settle. The research problem, the four research questions, the three hypotheses, and the control on which the study's value rests are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only systematic difference between the two implementations. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names, and the quantities the thesis quotes reach the page as macros regenerated from the raw logs. The readings are the author's too, including the restraints the evidence imposes: a sweep dominated by cross-thread handoff cannot bound an effect of the language, an unresolved comparison is not a demonstration of equivalence, and an operating point noticed after the data was in hand is described rather than tested.

How the output was checked. Model output was treated as a draft to be falsified, and all of it, prose and code alike, was read by the author before it entered the thesis. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. Most were then graded by how far the claim made of it could be traced: to the page or clause itself, to a publisher-deposited record carrying it, or no further than confirmed metadata behind a paywall. The audit records that grade per entry, so an entry whose supporting passage was never opened is listed rather than absorbed. It found four passages that overstated or misplaced their source, one a quotation absent from the work it was attributed to, and all four were corrected. The audit is kept beside the artefacts. Code was checked by running it against a conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in one predictable way: it will supply a plausible citation for a claim and a confident sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again in checking, and the defensible parts of this thesis are the parts

checked against a source, a specification, or a measurement.

Responsibility. The author has reviewed the whole of this thesis and is responsible for all of it. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[14] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. "Firecracker: Lightweight Virtualization for Serverless Applications". In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2020, pp. 419–434. URL: https://www.usenix.org/conference/nsdi20/presentation/agache.

[23] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[24] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[25] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[26] Cybersecurity and Infrastructure Security Agency. *Product Security Bad Practices*. Joint Guidance. Version 2.0. Bad practice 1 covers the development of new product lines in memory-unsafe languages and the memory-safety roadmap expectation, from which only products with an announced end-of-support date before 1 January 2030 are exempt. CISA and FBI, Jan. 2025. URL: https://www.cisa.gov/resources-tools/resources/product-security-bad-practices.

[27] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[28] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[29] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[30] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[31] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[32] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[33] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[34] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[35] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[36] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[37] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[38] C. Demichelis and P. Chimento. *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*. Request for Comments (RFC) 3393. Standards Track. Defines packet delay variation as the difference between consecutive one-way delays. Internet Engineering Task Force (IETF), Nov. 2002. DOI: 10.17487/RFC3393. URL: https://www.rfc-editor.org/rfc/rfc3393.

[39] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[40] The LLVM Project. *clang-tidy check `bugprone-use-after-move`*. Extra Clang Tools documentation. Accessed August 2026. 2026. URL: https://clang.llvm.org/extra/clang-tidy/checks/bugprone/use-after-move.html.

[41] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[42] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[43] W. Dietz, P. Li, J. Regehr, and V. Adve. "Understanding Integer Overflow in C/C++". In: *ACM Transactions on Software Engineering and Methodology* 25.1 (2015), 2:1–2:29. DOI: 10.1145/2743019. URL: https://doi.org/10.1145/2743019.

[44] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[45] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[46] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[47] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[48] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[49] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[50] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[51] R. Tang, M. Wang, X. Sun, L. Huang, Y. Huang, and X. Ma. "Converos: Practical Model Checking for Verifying Rust OS Kernel Concurrency". In: *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2025, pp. 1143–1159. URL: https://www.usenix.org/conference/atc25/presentation/tang.

[52] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[53] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[54] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[55] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[56] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu. "Rust Meets Linux: Lessons from an Evolving Experiment". In: *ACM Transactions on Computer Systems* 44.2 (2026). Extended version of the authors' USENIX ATC '24 study of Rust-for-Linux, 9:1–9:33. DOI: 10.1145/3789260. URL: https://doi.org/10.1145/3789260.

[57] Y. Fan, C. Wang, Z. Dong, T. Liu, and H. Wang. "An Empirical Analysis of Rust Integration in Android Open Source Project". In: *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. 2026, pp. 545–556. DOI: 10.1145/3803437.3805227. URL: https://doi.org/10.1145/3803437.3805227.

[58] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[59] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

[60] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[61] O. Pearce, J. Lange, and D. O'Keeffe. "RustMC: Automated Verification of Real-World Concurrent Rust". In: *Formal Techniques for Distributed Objects, Components, and Systems (FORTE)*. Lecture Notes in Computer Science. Springer, 2026, pp. 235–255. DOI: 10.1007/978-3-032-28187-6_14. URL: https://doi.org/10.1007/978-3-032-28187-6_14.

[62] W. Bugden and A. Alahmar. "Rust: The Programming Language for Safety and Performance". In: *arXiv preprint arXiv:2206.05503* (2022). DOI: 10.48550/arXiv.2206.05503. URL: https://doi.org/10.48550/arXiv.2206.05503.

[63] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[64] A. Levy, B. Campbell, B. Ghena, P. Pannuto, P. Dutta, and P. Levis. "The Case for Writing a Kernel in Rust". In: *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys)*. 2017, pp. 1–7. DOI: 10.1145/3124680.3124717. URL: https://doi.org/10.1145/3124680.3124717.

[65] C. Mo, L. Wang, S. Li, K. Hu, and B. Jiang. "Rust-Shyper: A Reliable Embedded Hypervisor Supporting VM Migration and Hypervisor Live-Update". In: *Journal of Systems Architecture* 142 (2023), p. 102948. DOI: 10.1016/j.sysarc.2023.102948. URL: https://doi.org/10.1016/j.sysarc.2023.102948.

[66] Y. Xiao, Y. Chen, X. Zhao, J. Duan, X. Zhang, W. Li, and X. Tao. "rpkt: A Generic, Safe, and Efficient Userspace Packet Processing Library in Rust". In: *Network and Parallel Computing (NPC 2024)*. Lecture Notes in Computer Science. 2025, pp. 454–465. DOI: 10.1007/978-981-96-2864-3_36. URL: https://doi.org/10.1007/978-981-96-2864-3_36.

[67] C. Cui and H. Xu. "Unleashing the Efficiency of Rust: An Empirical Study of Performance Bugs in Rust Projects". In: *Proceedings of the 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. 2025, pp. 371–381. DOI: 10.1109/ISSRE66568.2025.00045. URL: https://doi.org/10.1109/ISSRE66568.2025.00045.

[68] M. Naylor, A. Joannou, A. T. Markettos, P. Metzger, S. W. Moore, and T. M. Jones. "CHERI-SIMT: Implementing Capability Memory Protection in GPUs". In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2026, pp. 50–66. DOI: 10.1145/3760250.3762234. URL: https://doi.org/10.1145/3760250.3762234.

[69] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[70] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. "Stacked Borrows: An Aliasing Model for Rust". In: *Proceedings of the ACM on Programming Languages* 4.POPL (2020), 41:1–41:32. DOI: 10.1145/3371109. URL: https://doi.org/10.1145/3371109.

[71] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. "How Do Programmers Use Unsafe Rust?" In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA (2020), 136:1–136:27. DOI: 10.1145/3428204. URL: https://doi.org/10.1145/3428204.

[72] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim. "Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 84–99. DOI: 10.1145/3477132.3483570. URL: https://doi.org/10.1145/3477132.3483570.

[73] X. Zheng, Z. Wan, Y. Zhang, R. Chang, and D. Lo. "A Closer Look at the Security Risks in the Rust Ecosystem". In: *ACM Transactions on Software Engineering and Methodology* 33.2 (2024), 34:1–34:30. DOI: 10.1145/3624738. URL: https://doi.org/10.1145/3624738.

[74] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang. "Understanding and Detecting Real-World Safety Issues in Rust". In: *IEEE Transactions on Software Engineering* 50.6 (2024), pp. 1306–1324. DOI: 10.1109/TSE.2024.3380393. URL: https://doi.org/10.1109/TSE.2024.3380393.

[75] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[76] X. Cai, J. Liu, C. Liu, L. Bao, Y. Yu, and L. Jiang. "Fortifying the Seams Between C/C++ and Rust: Characterizing Bugs in Interop Tools". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3795532. URL: https://doi.org/10.1145/3795532.

[77] L. Schuermann, A. Thomas, and A. Levy. "Encapsulated Functions: Fortifying Rust's FFI in Embedded Systems". In: *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*. 2023, pp. 41–48. DOI: 10.1145/3625275.3625397. URL: https://doi.org/10.1145/3625275.3625397.

[78] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf. "Aliasing Limits on Translating C to Safe Rust". In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023), pp. 551–579. DOI: 10.1145/3586046. URL: https://doi.org/10.1145/3586046.

[79] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry. "Rust for Embedded Systems: Current State and Open Problems". In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024, pp. 2296–2310. DOI: 10.1145/3658644.3690275. URL: https://doi.org/10.1145/3658644.3690275.

[80] I. McCormack, T. Dougan, S. Estep, H. Hibshi, J. Aldrich, and J. Sunshine. "A Mixed Methods Study on the Implications of Unsafe Rust for Interoperation, Encapsulation, and Tooling". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3798277. URL: https://doi.org/10.1145/3798277.

[81] J. Hong and S. Ryu. "Automatically Translating C to Rust". In: *Communications of the ACM* 68.11 (2025), pp. 58–65. DOI: 10.1145/3737696. URL: https://doi.org/10.1145/3737696.

[82] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble. "Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency". In: *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. 2014, 9:1–9:14. DOI: 10.1145/2670979.2670988. URL: https://doi.org/10.1145/2670979.2670988.

[83] J. Huang, B. Mozafari, G. Schoenebeck, and T. F. Wenisch. "A Top-Down Approach to Achieving Performance Predictability in Database Systems". In: *Proceedings of the 2017 ACM International Conference on Management of*

Data (SIGMOD). 2017, pp. 745–758. DOI: 10.1145/3035918.3064016. URL: https://doi.org/10.1145/3035918.3064016.

[84] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[85] Y.-S. Liu, S. Qi, P.-Y. Lin, H.-S. Tsai, K. K. Ramakrishnan, and J.-C. Chen. "L^{2}5GC+: An Improved, 3GPP-Compliant 5G Core for Low-Latency Control Plane Operations". In: *Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. 2023, pp. 203–211. DOI: 10.1109/CloudNet59005.2023.10490024. URL: https://doi.org/10.1109/CloudNet59005.2023.10490024.

[86] C. Park and J. Nam. "CtrlBench: Towards Accurate API-Level Benchmarking in 5G Core Control Plane". In: *Proceedings of the 2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*. 2025, pp. 1832–1837. DOI: 10.1109/ICTC66702.2025.11388531. URL: https://doi.org/10.1109/ICTC66702.2025.11388531.

[87] L. Ardito, L. Barbato, R. Coppola, and M. Valsesia. "Evaluation of Rust Code Verbosity, Understandability and Complexity". In: *PeerJ Computer Science* 7 (2021), e406. DOI: 10.7717/peerj-cs.406. URL: https://doi.org/10.7717/peerj-cs.406.

[88] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[89] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[90] Z. A. Qazi, M. Walls, A. Panda, V. Sekar, S. Ratnasamy, and S. Shenker. "A High Performance Packet Core for Next Generation Cellular Networks". In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. 2017, pp. 348–361. DOI: 10.1145/3098822.3098848. URL: https://doi.org/10.1145/3098822.3098848.

[91] N. Larkin. *Alsoran: A 5G gNodeB Centralized Unit Written in Rust*. Open-source software repository. Proof of concept, development discontinued. Accessed August 2026. URL: https://github.com/nplrkn/alsoran.

Appendix A Key terms

Table A.1 collects the terms used throughout the thesis that are not acronyms, for readers outside the telecom domain. The acronyms are written out at first use and listed together in the List of acronyms on page v.

Table A.1: Key terms used throughout the thesis.

Term	Meaning
Control plane	The signalling and management path, as opposed to the user-plane data path
Tail latency	High-percentile latency (p99, p99.9, p99.99), the worst-case-oriented timing metric
Jitter	How much latency varies from one operation to the next, as opposed to its typical value
Saturation	The offered load beyond which the service can no longer keep up and requests start queueing
Shard	One of several independent slices of the UE state, each owned by exactly one worker thread
Open-loop	A load schedule whose send times are fixed in advance, so a slow response delays no later request
Coordinated omission	The bias an open-loop schedule avoids, in which a stalled client stops issuing requests and so never records the latency of the stall

Appendix B Literature search protocol

Section 3.2 reports the two findings this search produced. The protocol is recorded here so that the gap the study claims can be checked rather than taken on trust.

Candidate literature was gathered from the Association for Computing Machinery (ACM) Digital Library, the Institute of Electrical and Electronics Engineers (IEEE) Xplore, the USENIX Association, and Google Scholar, using combinations of the terms *Rust*, *C++*, *memory safety*, *tail latency*, *systems programming*, *performance comparison*, and *RAN / 5G control plane*. The search was restricted to peer-reviewed venues, except where an industry or government report is itself the object of study, as the memory-safety mandates and vendor vulnerability statistics are. The result set was then extended by backward and forward citation chaining from the core Rust-safety papers, and filtered to work that either measures a systems language empirically, formalises Rust's guarantees, or reports adoption experience. The domain literature (3GPP specifications and the O-RAN survey) was added separately to fix the architectural context.

Those sources alone would have missed the closest prior work, which is why two more were added. The vendor-hosted theses that come nearest to this study appear in none of the citation databases: not in Crossref, OpenAlex, the computer science bibliography DBLP, or Semantic Scholar. Neither forward nor backward chaining can reach them, because nothing indexed cites them and their own reference lists are not deposited. They were found instead by searching the institutional repositories directly, Digitala Vetenskapliga Arkivet (DiVA) for the Nordic universities and Finna for the Finnish ones. The same two searches establish the absence on the other side. Finna, which federates the Finnish university repositories, returns no thesis on Rust in a radio-access or telecom context. An author-affiliation search of Nokia's indexed publications in OpenAlex returns none either. Its entities hold on the order of thirty thousand indexed works, several hundred of which concern the radio access network, and one mentions Rust, in an unrelated preprint on trusted-execution monitoring.

One practical note belongs with the protocol, since it distorts a naive search. Free-text queries for *Rust* in the general indexes rank plant pathology and corrosion chemistry above the programming language, so field-restricted or discipline-restricted queries are necessary.

The four strands of Section 2.2 leave this thesis in the following positions, which Section 2.3 then answers against the studies closest to it.

The performance strand is settled on the average case. Compiled systems languages sit close on time and energy [62, 63], Rust's safety is cheap at run time [12, 64], and that holds where timing is tightest, in embedded hypervisors and in packet processing within a few percent of a C reference [45, 65, 66]. Where a cost does appear it has been pinned down: to bounds, overflow and division checks, to that same check set against hardware-enforced capabilities, and to recurring anti-patterns [46, 47, 67]. One like-for-like port puts the bounds-check share at 34% of a 46% geometric-mean overhead, and cautions that code credited with cheap checks has sometimes declared its hot functions `unsafe` [68]. That caveat does not reach the `unsafe`-free core compared here. None of this work reports

behaviour past a saturation point.

The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [11, 69, 70], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [27, 48, 71], auditing that finds defects at scale even so and unsafety spreading through dependency chains [49, 72, 73, 74], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [8, 9, 17, 19, 20, 75].

The interoperability strand qualifies the adoption argument, and Section 2.2 states its central limit: confining `unsafe` to a seam does not protect the Rust side, because foreign code in the same address space can corrupt Rust-owned data anyway [58, 59]. The seam has costs of its own. It carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [56, 57, 60, 76, 77, 78, 79, 80]. Automating the rewrite is not a solved problem either, since current translators still emit code whose safety the compiler cannot validate [81]. That is why the migration cost measured here is the cost of a hand-written seam.

The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [6, 7, 82, 83], thread-per-core structuring moving the tail in its own right [84], O-RAN disaggregation [3], control-plane latency as an active target in the core network too [85, 86], adoption evidence pointing at the learning curve rather than run-time cost [54, 55, 87], and standard guidance on repeat-based evaluation and fault injection [88, 89]. Rust has mobile-network precedent in the datapath [90] and, on the control-plane side, an open-source gNB-CU with a separated CU-CP and CU-UP over E1. That last one is an existence proof and not a comparison, reporting no latency and carrying no C++ counterpart built to the same specification [91].

Appendix C Fault-injection example

As an illustration of the fault-injection study (Chapter 4, RQ2), the use-after-free defect is shown in both languages. In C++ the alias into the context map survives the `erase` that frees it, so the following program compiles cleanly and is flagged only at runtime, by AddressSanitizer and ThreadSanitizer:

```
std::unordered_map<unsigned long, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001);  // alias into the container
ctxs.erase(1001);               // frees the node that p refers to
p->messages_handled += 1;       // use-after-free
```

The analogous safe-Rust program does not compile: the borrow checker forbids mutating the map (`remove`) while the reference `p` into it is still live, so the defect cannot be written in the first place (error `E0499`):

```
let mut ctxs: HashMap<u64, UeContext> = HashMap::new();
ctxs.insert(1001, UeContext { messages_handled: 0 });
let p = ctxs.get_mut(&1001).unwrap(); // mutable borrow starts
ctxs.remove(&1001);                   // borrow while p live -> rejected
p.messages_handled += 1;              // p used after the remove
```

Appendix D Statistical design: family membership and resolving power

Section 3.5 fixes which comparisons enter the corrected family and how far into a difference the design can see. Three points behind those commitments are recorded here rather than in the running text.

Why an unresolvable comparison is not free. Testing achieved throughput below saturation would re-measure the operating point and not the code. It would also cost the comparisons that can resolve something, because Holm divides its budget across the whole family and a test that separates nothing still takes a share of it. Above saturation the same reasoning runs the other way. What each variant still sustains is pinned to nothing there, so excluding it would discard the only observable overload offers.

Why no percentile is tested in overload. Under a fixed intended send schedule, a producer that cannot keep up falls behind cumulatively, so a percentile measured there reports accumulated backlog against a window that cuts it off. Attaching a p-value to a lower bound would test the harness and not the software.

The floors the design can reach. An exact test on n pairs cannot return a two-sided p below $2/2^n$, and after correcting a family of m comparisons the smallest attainable adjusted value is m times that. That is where the 0.0000019 and 0.000027 of Section 3.5 come from. A campaign whose corrected floor lies above the threshold being applied cannot demonstrate significance however large the effect, so reporting "not significant" from such a campaign would say more about its repeat count than about the languages.

Appendix E Host and campaign configuration

Table E.1 itemises the three machines the campaigns ran on: the bare-metal host of Section 4.1 that every performance and resource table of the main sweep is drawn from, the virtualised host it replaced, and the second bare-metal host at a different design read against it. The resident-set decomposition belongs to the second bare-metal host. The fault catalogue was compiled and run on the development machine, whose toolchain its own caption records. Of the adoption-cost axes only build time is host-sensitive, which is why that one is reported as a paired ratio while the footprint and dependency figures are deterministic. The three small paired experiments predate the move to bare metal and carry no environment record, so each supports a ratio between two arms measured alternately in one session. Section 4.3 points here for the field-by-field record of what each campaign captured. Every column reports what was measured, and the protocol they are read against is published as `bench/TESTBED.md`. Table E.2 gathers the configuration of the service, the workload and the campaign.

What each campaign recorded. Each of the 1120 runs of the factorial design in Table E.2 contributes one newline-delimited JSON record. Nothing is aggregated at capture time. A record carries the configuration (language, allocator, workload, shard and producer counts, target rate), the outcome (operations completed, errors, wall-clock duration, achieved throughput), the latency distribution from the HDR histogram (mean, standard deviation, the quartiles, p50, p90, p99, p99.9, p99.99 and the maximum, in nanoseconds), a shorter block for inter-operation delay variation (mean, standard deviation, p50, p99, p99.9, the maximum and a sample count), the resource counters from `getrusage` (peak resident set size, user and system CPU seconds, CPU utilisation), and the per-operation-kind counts. A `jemalloc` arm adds that allocator's own accounting. A system-allocator arm stores those fields as unavailable rather than as zero, on the same principle as the governor and turbo state. The kind counts are what makes a divergence between the languages show up as a parity failure instead of a performance difference. Each run also persists its complete HDR bucket contents to a sidecar, so the repeats can be pooled before a far-tail percentile is read, and a separate file records `perf` counters at two scopes, the end-to-end path and the operation on its own.

The three small paired experiments have a different shape. For each of 15 interleaved repeats the decomposition records the batch-mean cost in both languages, once for the operation called directly and once through the threaded service. A third dataset of the same shape over 20 repeats records sustained throughput through the bounded queue and completion primitives alone (Section 4.4.2). The FFI comparison of Section 4.4.5 has that structure over 15 repeats. All three predate the environment record, which is why each is read as a ratio between its two arms.

Build time is the one RQ4 axis that is a timing measurement. It is recorded over 15 interleaved repeats at 14 parallel jobs on the development machine, alternating the toolchains so the ratio survives a change of machine even though the seconds do not. The static footprint and the resolved dependency graph are read from the linked binaries and the

Table E.1: The three machines the campaigns ran on, each reported as measured. Every hardware, kernel and toolchain cell is generated from that campaign's own environment capture rather than restated from the protocol of `bench/TESTBED.md`. The last two rows are prose, because what an instrument set is adequate for is a judgement and not a captured field. The columns are arranged so that each adjacent pair isolates one change, with the exceptions the Kernel and Toolchain rows record. Reported sweep against virtualised host: the same design and the same container image on a different machine, one `g++` release apart within GCC 15, which is where the hardware effect is read from. Reported sweep against second bare-metal host: a different shard count, load grid, repeat count and compiler on comparable hardware, which is what tests whether the findings depend on those choices. Rows where all three agree describe the source or the method rather than any machine. The *Short of protocol* marks are deliberate and are argued where they appear.

Aspect	Reported sweep	Virtualised host, same design	Second bare-metal host, different design
Host	AMD EPYC 7443, 24 cores, 1 socket	Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs	AMD EPYC 7443P, 24 cores, 1 socket
Kernel	6.17.0-23-generic	7.0.2-1.el8.elrepo.x86_64	6.17.0-23-generic
Hypervisor	none. Governor performance, boost off, verified on the machine	kvm. Frequency and turbo host-controlled, unobservable from inside	none. Governor performance, verified on the machine
CPU steal	None: no hypervisor to lose a timeslice to	Not characterised. The capture's single one-second sample read 0, and no steal probe was run on this host	0 of 60 intervals on 16 pinned cores
Isolation	`nohz_full` 2-17. `isolcpus` none, omitted on purpose	`isolcpus` none, `nohz_full` none	`isolcpus` none, `nohz_full` none. *Short of protocol*
Hugepages	madvise	always, and only the system-allocator arm inherits it	madvise
Toolchain	`g++ 15.2.1`, `rustc 1.97.1`, pinned in a container image	`g++ 15.1.1`, `rustc 1.97.1`	`g++ 13.3.0`, `rustc 1.98`, so a compiler change is tested too
Pinning	Cores 2–17, 7 shards, 7 producers	Cores 2–15, 7 shards, 7 producers	Cores 2–17, 8 shards, 8 producers
SMT	off, verified	Absent from the topology the guest is shown, and decided on the host	off, verified
Repeats	20 over 14 loads, 1120 runs	20 over 14 loads, 1120 runs	15 over 8 loads, 480 runs. *Short of protocol*
Instruments	Counters, pooled high dynamic range (HDR) sidecars, jitter, allocator accounting	None available in the guest	Counters at two scopes, HDR sidecars, allocator accounting
Adequate for	Every result reported here	Everything except the far tail below saturation	Confirming each finding survives a change of design

manifests.

The fault catalogue is categorical. Each of the 14 cases yields four tool logs (AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer on the C++ program, and `rustc` on the safe-Rust counterpart) and one taxonomy row carrying the three sanitiser outcomes, the detection stage, the compiler result, and the expectation registered before

Table E.2: Configuration of the service under test, the workload driving it, and the measurement campaign. Service parameters are fixed by the frozen specification, with one exception: the shard count is set to match the cores the host offers (Section 4.1). All of them, that one included, are identical in both languages. Campaign figures are generated from the measurement logs together with the tables that report them. The representative load is fixed rather than derived, for the reason Section 4.2 gives, and the throughput plateau is the lowest of the four variants' own plateaus, which differ.

Parameter	Value	Note
Service under test, identical in both languages		
Shards, and worker threads	7	one thread owns one shard. Identical in both languages, which is what the fairness control requires
Queue bound per shard	1 024 requests	bounded and blocking, so overload is observable
Initial map capacity	4 096 entries	reserved so steady state does not rehash
Bearers per context	at most 8	
Signaling payload	64 bytes	
Context ceiling per shard	200 000	
Shard selection	SplitMix64, then modulo	pinned: diffuses sequential identifiers
Intra-shard hash	FNV-1a, 64-bit	pinned on both sides and checked bit-for-bit
Workload		
Workloads	signal, mixed	mixed walks the full RAN lifecycle
Active UEs	5 000	
Offered-load levels	14	up to 784 kops/s
Representative load	56 kops/s	fixed, see caption
Measured throughput plateau	392 kops/s achieved	the lowest of the four, reached at an offered 392 kops/s. The Rust arms track one rung further, to 448
Load model	open loop, fixed intended schedule	latency measured from intended send time
Campaign		
Build variants	4	two languages × two allocators
Repeats per sweep point	20	variants interleaved within each repeat
Measured runs	1120	
Latency capture	HDR histogram, [1 ns, 3.6 s], 3 s.f.	same algorithm and configuration on both sides
Summary statistic	median with bootstrap 95% confidence interval	intervals reported in full, not as half-widths
Language comparison	exact paired permutation test	14 tests, Holm-corrected
Attainable p floors	0.0000019 / 0.000027	uncorrected / after correction

the study. The two original boundary cases yield a second table from the model checkers, and the `unsafe` one a third from the interpreter. The table and figure in Section 4.4.4 are generated from those logs.

Every load sweep writes the environment record of Section 3.6 beside its results, including the fields a given host does not expose. The governor and turbo state are stored as unavailable rather than left out, so the limitation of Section 4.1 is a property of the dataset. The virtualised campaign shares the sweep's design and predates the later instruments: its records carry the percentiles and the resource counters and nothing else. That absence is

the measurement-validity argument itself, which is why the campaign is kept. The second bare-metal campaign carries the same record shape as the reported sweep, including the allocator accounting that decomposes the resident set, but runs a different shard count, load grid, repeat count and toolchain. Each campaign has its own generator and macro prefix.

The tables that follow are the measurements the main text cites from this appendix.

Table E.3: Inter-operation delay variation at a sustained offered load of 56 kops/s, in microseconds, as a median over 20 interleaved repeats with a bootstrap 95% interval. Measured within one producer and never across producers, since two producers share no schedule and a difference between them is not a variation either one experienced. Reported for completeness rather than as evidence: no research question names this metric, and it is absent from the paired family of Table 4.2.

Variant	p50	p99	p99.9	max
C++ (system alloc)	1.5	19.4	44.3	169.9
	[1.3, 1.6]	[13.9, 23.1]	[42.7, 47.3]	[165.8, 202.9]
C++ (jemalloc)	1.5	23.4	44.0	173.8
	[1.4, 1.6]	[18.1, 31.3]	[41.7, 46.0]	[154.9, 185.9]
Rust (system alloc)	1.4	25.1	45.1	106.7
	[1.2, 1.5]	[17.8, 31.7]	[42.4, 46.6]	[60.7, 113.0]
Rust (jemalloc)	1.7	25.4	43.8	74.7
	[1.6, 1.8]	[19.7, 29.7]	[41.1, 46.2]	[57.8, 115.3]

Table E.4: Where the peak resident set goes on the second bare-metal host, from `jemalloc`'s own accounting in each of the 240 runs of the two `jemalloc` arms. Medians in MiB over the steady-state loads, with the paired language difference in the last column. The two rows below the rule are the argument: the allocator holds *less* in the Rust build, and the process holds more, so the difference is outside the allocator. The final row is the quantity the other rows decompose. The derived row is the median of the per-run difference and not the difference of two medians, because the two figures it subtracts come from one process at one moment, so the column does not sum exactly.

Component	C++	Rust	Difference
Live allocation (`allocated`)	0.78	0.62	−0.16
In active pages (`active`)	1.43	0.79	−0.64
Arena bookkeeping (`metadata`)	4.19	6.26	+2.07
Resident in allocator mappings (`resident`)	11.02	9.10	−1.92
Outside those mappings (derived)	4.04	7.86	+3.82
Peak resident set (`getrusage`)	15.08	16.82	+1.74

The allocation profile on the second bare-metal host was taken to name the call sites behind the resident-set difference between the two builds. Two departures from the campaign explain why it cannot. The first profile ran closed loop at 455 kops/s, past the plateau bracketed at 448–512 kops/s. Repeating it at a swept load below saturation shrank the discrepancy by most of its size and did not remove it. What remains is that profiling requires an allocator built to sample every allocation and record a backtrace, so the arm under profile is not the arm the campaign measured. Each symbolised profile puts almost all of its flat bytes on a frame that allocates nothing, 98.4% on a deallocation symbol in the C++ arm and 100.0% on the allocator's own backtrace routine in the Rust one. Attributing the footprint would need a profile taken at a swept load, on a build whose peak RSS reproduces the campaign's ordering.

Appendix F Implementation of the two cores

Section 3.7 records the three divergences that carry a measured consequence. This appendix fills in the constructions those paragraphs summarise.

Layout policy accounts for the 8-byte gap on `UeContext`. C++ guarantees that non-static data members sit in declaration order, so the narrow fields the specification places between wider ones open two holes of alignment padding. Rust carries no `#[repr(C)]` annotation here, so it packs the small scalars together and both holes close. Reads return a `UeContextView` copied out of the worker, not a reference into live shard state, which keeps the shared-nothing guarantee intact.

The hashing pins exist because the defaults differ too much to leave in place. C++'s `std::hash<uint64_t>` is usually the identity function, while Rust's default `SipHash-1-3` is cryptographically strong and correspondingly slower. Keeping both would have measured a standard-library choice, not a language. Shard assignment runs SplitMix64 first because the workload draws sequential identifiers, and a naive modulo would map those onto shards in a fixed repeating pattern. Inside the shard both maps use FNV-1a over the eight little-endian bytes of the identifier, and a test checks that the Rust `BuildHasher` produces a digest bit-identical to the reference `fnv1a_64`.

The C++ bounded queue is short. It uses the predicate form of `wait` so that spurious wakeups are handled correctly, and it releases the lock before notifying. The point is that it exists at all. Every one of those 51 lines is a place where a concurrency defect could enter: that the two condition variables are never confused, that the predicates match the notifications, that no wakeup is lost. This is the same asymmetry the fault-injection study measures, showing up here in ordinary code with no defect in it.

A C++ `Request` always carries the create parameters, the 68-byte signalling message, the bearer entry, and the reconfiguration parameters, whether the operation in flight uses them or not. Unused fields are default-initialised and moved along with the used ones. The caller recovers its typed result with `std::get<OpResult<T>>`, which throws `std::bad_variant_access` if the worker ever writes an alternative the caller did not expect. That exception cannot fire in the code as it stands, because each `OpKind` is handled by exactly one branch. The invariant is upheld by reading two files, not by the compiler. Starting the service allocates the shards, spawns one worker thread per shard, and hands each worker exclusive ownership of its store. Shutdown pushes the `Stop` sentinel already named in Section 3.7, then joins the workers. Both languages make teardown idempotent. An operation submitted outside the running window is a caller error, reported out of band so it does not widen the closed `OpError` set of Section 3.7. C++ throws `std::logic_error` and Rust panics. C++ makes `Request` move-only, a property it picks up from the move-only `std::promise` it contains. The only shared mutable state is the atomic activity counter. C++ holds it by reference, Rust as a cloned `Arc<AtomicU64>`, and both increment it with relaxed ordering because nothing branches on its value.

Appendix G Engineering guidelines for incremental adoption

Section 5.4 keeps the two guidelines the study arrived at late. The seven below are the practical form of the rest of the evidence, in the order a first migration would run them. Each is tied to the finding that supports it. A guideline quoted without its limit is how the RQ2 result becomes "the migrated component is safe" and the RQ4 result becomes "migration is cheap", and this study establishes neither.

Choose the first component for its interface, not its size. The boundary's runtime cost is bounded at 10.91% of an operation (RQ4), so the criterion for a first Rust component is not how small or how hot it is. It is how narrow and how scalar its interface is. The seam here stayed at 11 entry points because the subsystem's API was already lifecycle-oriented, typed, and free of shared mutable structures crossing the boundary. A component whose interface passes rich C++ objects, templates, or callbacks will need those designed out first, and that redesign is the schedule risk, not the FFI.

Put the ABI in its own crate and keep the core `unsafe`-free. Concentrating every raw-pointer operation in a separate seam crate that depends on an unmodified core (Section 4.4.5) is worth adopting as a rule, not treating as an accident of this prototype. It keeps the dependency-propagation problem Evans et al. describe [49] away from the safety-critical code, and it makes the reviewable surface of the migration a fixed, countable thing. Two qualifications belong with it. It bounds what has to be *reviewed*, not what can be *corrupted*. While the host remains C++ the two share a heap, so a team needing the stronger property has to look at in-process isolation [58, 59] and not at interface discipline alone. And the generated part of a seam is not above suspicion, because binding generators have their own defect history [76]. Review it as code instead of trusting it as tooling output.

Do not re-budget concurrency review effort on the strength of memory safety. Adopting Rust removes use-after-free, double-free, iterator invalidation, and data races from the defect budget entirely, and those are the classes that dominate the security statistics [8, 9]. It removes nothing from the deadlock and atomicity-violation budget. Lock-ordering discipline, invariants about which locks may be held simultaneously, and review for composed-but-not-atomic operations remain exactly as necessary as they were in C++.

Prefer a shared-nothing structure to a shared-and-locked one, in either language. The strongest concurrency result in this study belongs to neither language. The thread-per-shard design removes locks from the hot path altogether, which is why the boundary defects had to be built as standalone programs instead of found in the implementations. Where state can be partitioned by a natural key, as UE state can be partitioned by `ue_id`, partitioning it turns a class of concurrency defects into a structural impossibility for both languages, and it does so without either compiler's help.

Budget for the learning curve, and measure it. The dominant cost identified here is human, which matches what adoption studies report [54]. This thesis cannot quantify that cost, because it is a single-author study, so the guideline is correspondingly modest. Treat the first component as a training investment with a deliberately generous schedule, and instrument it, recording review time and defect density, so the second component can be planned on measured effort instead of assumed effort.

Measure the component, not the round-trip, and do not generalise either. An end-to-end request/response measurement of a design like this one is 0.250% language and the rest scheduler (Section 4.4.1). Below saturation it will therefore return "no difference" between almost any two competent implementations. A vendor running its own comparison should budget for both: the end-to-end measurement, which says what a deployment will feel, and the component measurement, which says what the language is doing.

Sweep past saturation, because that is where the languages stopped agreeing. Every conclusion drawn about performance while the sweep stopped at nominal load was a null. Each was correct at that operating point and silent about the one where the two implementations differ by 1.7× (Section 4.4.2). The knee is the region that decides whether a signalling storm degrades service or ends it. Sweeping past it costs one more afternoon of machine time. The decision the framework should inform is "measure this on your component, and measure it past the point where your component stops coping", not "Rust is fast enough, proceed".